Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Изучение архитектуры и принципов работы сверточных нейронных сетей (CNN) для задач компьютерного зрения.

Цель работы: освоение теоретических основ и приобретение практических навыков построения сверточных нейронных сетей.

Репозиторий: https://github.com/bickrosss/NN_lab3

Порядок выполнения работы:

Задание 1. Создать нейронную сеть, распознающую рукописные цифры. Используя подготовленную базу и шаблон ноутбука, нормировать данные, создать и обучить сверточную сеть.

Сеть должна содержать минимум 2 сверточных слоя, полносвязные слои, слои подвыборки, нормализации, регуляризации по 1 шт. Функция ошибки – категориальная кроссэнтропия, оптимизатор – Adam с шагом обучения 0.001, размер батча – 128, количество эпох 15, детали обучения – отображать.

Вывести график обучения: доли верных ответов на обучающей и проверочной выборках.

1.1 Реализация средствами библиотеки Keras.

1.1.1 Подключение необходимых для работы библиотек и загрузка датасета.

Для решения задачи распознавания рукописных цифр используется датасет MNIST из библиотеки Keras. Подключаются необходимые модули для работы с массивами, отрисовки графиков, генерации случайных чисел и обработки изображений.

```
# загрузка датасета MNIST
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Рисунок 1.1.1 – Результат загрузки датасета MNIST

```
# Подключение утилит для to_categorical
from tensorflow.keras import utils

# Подключение библиотеки для работы с массивами
import numpy as np

# Подключение библиотек для отрисовки изображений
import matplotlib.pyplot as plt

# Подключение библиотеки для генерации случайных чисел
import random

# Подключение класса для работы с изображением
from PIL import Image

# Вывод изображений в ноутбуке, а не в консоли или файле
%matplotlib inline
```

Рисунок 1.1.2 – Подключение библиотек

1.1.2 Визуализация изображений из датасета.

Для ознакомления с содержимым датасета выводится по одному случайному изображению из каждого класса (цифры от 0 до 9). Это позволяет убедиться в корректности загрузки данных и оценить характер исходных изображений.

Листинг 1.1.1 – Вывод изображения каждого класса

```
# вывод изображений каждого класса для ознакомления с датасетом
fig, axs = plt.subplots(1, 10, figsize=(25,3))
for i in range(10):
    label_indexes = np.where(y_train == i)[0]
    index = random.choice(label_indexes)
    img = x_train[index]
    axs[i].imshow(Image.fromarray(img), cmap='gray')

plt.show()
```

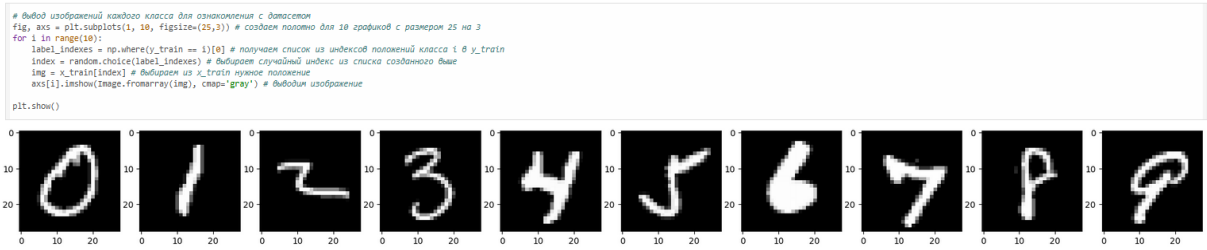


Рисунок 1.1.3 – Изображения из датасета MNIST

1.1.3 Подготовка данных к обучению.

Перед подачей данных в сверточную нейронную сеть необходимо изменить размерность массивов изображений: добавить дополнительную ось, обозначающую количество цветовых каналов. Для монохромных изображений MNIST это значение равно 1. Метки классов преобразуются в формат One-Hot Encoding, что позволяет использовать функцию ошибки категориальной кроссэнтропии.

```
# добавляем размерность массиву mnist, чтобы сеть поняла что это ЧБ
x_train = x_train.reshape(x_train.shape[0], x_train.shape[1], x_train.shape[2], 1)
x_test = x_test.reshape(x_test.shape[0], x_test.shape[1], x_test.shape[2], 1)

# преобразуем выборки с ответами в ОНЕ
y_train = utils.to_categorical(y_train, 10)
y_test = utils.to_categorical(y_test, 10)

# посмотрим форматы выборок перед обучением
print('x_train:', x_train.shape)
print('x_test:', x_test.shape)
print()
print('y_train:', y_train.shape)
print('y_test:', y_test.shape)
```

```
x_train: (60000, 28, 28, 1)
x_test: (10000, 28, 28, 1)

y_train: (60000, 10)
y_test: (10000, 10)
```

Рисунок 1.1.4 – Размерности выборок датасета

1.1.4 Создание архитектуры сверточной нейронной сети.

Создаваемая модель удовлетворяет всем требованиям задания. Сеть содержит два сверточных блока, в каждом из которых присутствуют: сверточный слой, активация LeakyReLU, слой пакетной нормализации (BatchNormalization), слой подвыборки (MaxPooling2D) и слой регуляризации (Dropout). Для нормализации входных данных используется встроенный слой Rescaling, который приводит значения пикселей из диапазона [0, 255] в [0, 1]. После сверточных блоков идёт полносвязная часть с активацией LeakyReLU, нормализацией и регуляризацией.

Листинг 1.1.2 – Архитектура нейронной сети

```
# Подключение необходимых слоев
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, \
                                    Dropout, BatchNormalization, LeakyReLU, Rescaling
from tensorflow.keras.optimizers import Adam

# Создание модели
model = Sequential()

# Слой нормализации входных данных
# Rescaling приводит значения пикселей из диапазона [0, 255] в [0, 1]
model.add(Rescaling(1./255, input_shape=(28, 28, 1)))

#  Первый сверточный блок
model.add(Conv2D(32, (3, 3), padding='same'))
model.add(LeakyReLU())
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))   # 28x28 -> 14x14
model.add(Dropout(0.25))

#  Второй сверточный блок
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(LeakyReLU())
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))   # 14x14 -> 7x7
model.add(Dropout(0.25))

#  Полносвязная часть
model.add(Flatten())

model.add(Dense(256))
model.add(LeakyReLU())
```

```
model.add(BatchNormalization())
model.add(Dropout(0.5))

# Выходной слой: 10 нейронов (цифры 0-9), активация softmax
model.add(Dense(10, activation='softmax'))

# Вывод архитектуры модели
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessing/data_layer.py:95
he model instead.
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
leaky_re_lu (LeakyReLU)	(None, 28, 28, 32)	0
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
dropout (Dropout)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 64)	0
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_1 (Dropout)	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
dense (Dense)	(None, 256)	803,072
leaky_re_lu_2 (LeakyReLU)	(None, 256)	0
batch_normalization_2 (BatchNormalization)	(None, 256)	1,024
dropout_2 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 10)	2,570

```
Total params: 825,866 (3.15 MB)
Trainable params: 825,162 (3.15 MB)
Non-trainable params: 704 (2.75 KB)
```

Рисунок 1.1.5 – Структура созданной нейронной сети

1.1.5 Компиляция и обучение модели.

В соответствии с заданием в качестве функции ошибки используется категориальная кроссэнтропия, в качестве оптимизатора – Adam с шагом обучения 0.001. Размер батча – 128, количество эпох обучения – 15. Параметр verbose=1 обеспечивает отображение деталей обучения в процессе работы.

```
In [6]:
# Компиляция модели
# Функция ошибки - категориальная кроссэнтропия
# Оптимизатор - Adam с шагом обучения 0.001 (одна тысячная)
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=0.001),
    metrics=['accuracy']
)

# Обучение модели
# Размер батча - 128, количество эпох - 15, детали обучения отображаются (verbose=1)
history = model.fit(
    x_train, y_train,
    batch_size=128,
    epochs=15,
    validation_data=(x_test, y_test),
    verbose=1
)
```

```
Epoch 1/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 17s 19ms/step - accuracy: 0.9416 - loss: 0.1915 - val_accuracy: 0.4431 - val_loss: 1.7301
Epoch 2/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9743 - loss: 0.0822 - val_accuracy: 0.9862 - val_loss: 0.0394
Epoch 3/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9801 - loss: 0.0637 - val_accuracy: 0.9866 - val_loss: 0.0376
Epoch 4/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9830 - loss: 0.0561 - val_accuracy: 0.9892 - val_loss: 0.0302
Epoch 5/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9826 - loss: 0.0524 - val_accuracy: 0.9890 - val_loss: 0.0353
Epoch 6/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9846 - loss: 0.0462 - val_accuracy: 0.9894 - val_loss: 0.0305
Epoch 7/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9862 - loss: 0.0427 - val_accuracy: 0.9893 - val_loss: 0.0312
Epoch 8/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9871 - loss: 0.0415 - val_accuracy: 0.9893 - val_loss: 0.0287
Epoch 9/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9877 - loss: 0.0368 - val_accuracy: 0.9904 - val_loss: 0.0281
Epoch 10/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9885 - loss: 0.0345 - val_accuracy: 0.9897 - val_loss: 0.0313
Epoch 11/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9889 - loss: 0.0331 - val_accuracy: 0.9911 - val_loss: 0.0273
Epoch 12/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9895 - loss: 0.0314 - val_accuracy: 0.9915 - val_loss: 0.0282
Epoch 13/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9901 - loss: 0.0306 - val_accuracy: 0.9902 - val_loss: 0.0291
Epoch 14/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9910 - loss: 0.0285 - val_accuracy: 0.9919 - val_loss: 0.0241
Epoch 15/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9905 - loss: 0.0295 - val_accuracy: 0.9879 - val_loss: 0.0329
```

Рисунок 1.1.6 – Процесс обучения модели

Из выводимых в процессе обучения метрик видно, что модель быстро достигает высокой точности уже после первой эпохи (≈98% на проверочной выборке). К пятнадцатой эпохе точность на обучающей выборке составляет 99.05%, на проверочной – 98.79%, что говорит об успешном обучении модели без признаков переобучения.

1.1.6 Построение графика обучения.

Для визуальной оценки процесса обучения строятся два графика: динамика доли верных ответов (accuracy) и динамика функции ошибки (loss) на обучающей и проверочной выборках в зависимости от номера эпохи.

Листинг 1.1.3 – Построение графика обучения

График обучения

Доли верных ответов на обучающей и проверочной выборках

```
plt.figure(figsize=(12, 5))

# График accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Обучающая выборка')
plt.plot(history.history['val_accuracy'], label='Проверочная выборка')
plt.title('Доля верных ответов (accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# График Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Обучающая выборка')
plt.plot(history.history['val_loss'], label='Проверочная выборка')
plt.title('Функция ошибки (loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
In [7]: # ========== График обучения ==========
        # Доли верных ответов на обучающей и проверочной выборках

        plt.figure(figsize=(12, 5))

        # График accuracy
        plt.subplot(1, 2, 1)
        plt.plot(history.history['accuracy'], label='Обучающая выборка')
        plt.plot(history.history['val_accuracy'], label='Проверочная выборка')
        plt.title('Доля верных ответов (accuracy)')
        plt.xlabel('Эпоха')
        plt.ylabel('Accuracy')
        plt.legend()
        plt.grid(True)

        # График loss
        plt.subplot(1, 2, 2)
        plt.plot(history.history['loss'], label='Обучающая выборка')
        plt.plot(history.history['val_loss'], label='Проверочная выборка')
        plt.title('Функция ошибки (loss)')
        plt.xlabel('Эпоха')
        plt.ylabel('Loss')
        plt.legend()
        plt.grid(True)

        plt.tight_layout()
        plt.show()
```

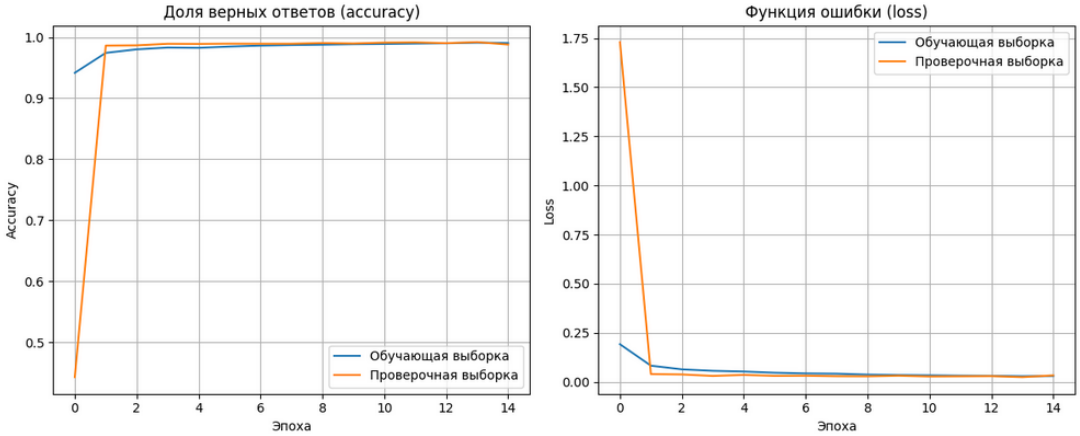


Рисунок 1.1.7 – График процесса обучения модели

На графике видно, что доля верных ответов на обоих наборах данных стабильно растёт и быстро выходит на плато около 99%. Кривые ошибки также демонстрируют успешное обучение: после резкого падения на первой эпохе значения функции потерь продолжают плавно снижаться, без расхождения между обучающей и проверочной выборками, что свидетельствует об отсутствии переобучения.

1.1.7 Построение матрицы ошибок.

Для более детального анализа качества классификации строится нормализованная матрица ошибок. Она показывает, какая доля изображений каждого класса была верно распознана моделью, а в каких случаях возникли ошибки. Также вычисляется общая точность работы модели на тестовой выборке.

Листинг 1.1.4 – Построение матрицы ошибок

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
# Получаем предсказания модели на тестовой выборке
y_pred = model.predict(x_test)

# Преобразуем OHE в номера классов
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)

# Строим матрицу ошибок (нормализованную)
cm = confusion_matrix(y_true_classes, y_pred_classes, normalize='true')
cm = np.round(cm, 3)

# Отображаем матрицу ошибок
fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=np.arange(10))
disp.plot(ax=ax, cmap='Greens', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

# Вывод общей точности
accuracy = np.mean(y_pred_classes == y_true_classes)
print(f'\nОбщая точность на тестовой выборке: {accuracy:.4f}
({accuracy*100:.2f}%)')
```

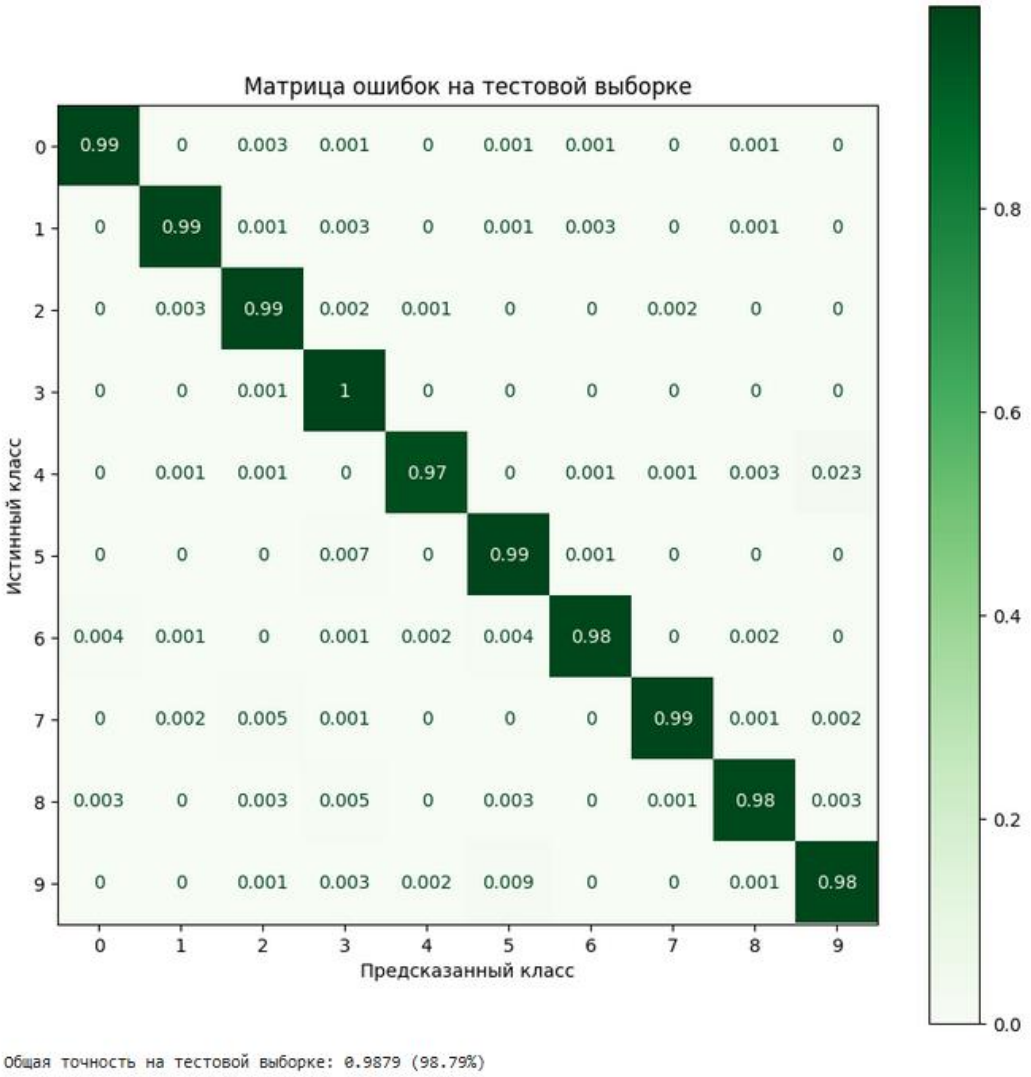


Рисунок 1.1.8 – Матрица ошибок на тестовой выборке

Анализ матрицы ошибок показывает, что для большинства классов доля верных предсказаний составляет 98-99%. Наименьшая точность наблюдается для классов 4 и 9 (97-98%) – модель иногда путает их между собой, что объясняется визуальной схожестью этих цифр в рукописном виде. В целом общая точность распознавания на тестовой выборке составила 98.79%, что является высоким результатом и подтверждает корректность построенной архитектуры и выбранных гиперпараметров.

1.2 Реализация той же задачи распознавания рукописных цифр с использованием библиотеки PyTorch.

В рамках расширения работы та же задача распознавания рукописных цифр MNIST была реализована средствами библиотеки PyTorch. Архитектура сети, гиперпараметры обучения и итоговые метрики совпадают с

требованиями исходного задания. Цель данного раздела – освоить альтернативный фреймворк глубокого обучения и сравнить его с Keras на одной задаче.

1.2.1 Подключение необходимых библиотек.

В отличие от Keras, в PyTorch модули импортируются по отдельности: torch для основных операций, torch.nn для слоёв нейронной сети, torch.optim для оптимизаторов, torch.utils.data для загрузчиков данных, torchvision для готовых датасетов и преобразований. Также определяется устройство, на котором будет производиться обучение (GPU при наличии, иначе CPU).

Загрузка библиотек

In [1]:

```
# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Базовые библиотеки
import numpy as np
import matplotlib.pyplot as plt
import random

# Метрики
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Вывод изображений в ноутбуке
%matplotlib inline

# Определение устройства (GPU если доступен)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Используется устройство: {device}')
```

```
Используется устройство: cuda
```

Рисунок 1.2.1 – Подключение библиотек и определение устройства

1.2.2 Загрузка датасета MNIST.

В PyTorch нормализация данных выполняется на этапе загрузки через объект transforms.Compose. Преобразование ToTensor() автоматически приводит значения пикселей к диапазону [0, 1] (выполняет ту же роль, что и Rescaling в Keras), а Normalize дополнительно центрирует данные по среднему и стандартному отклонению, рассчитанным для датасета MNIST.

Загрузка датасета MNIST

```
In [2]:
# Преобразования: тензор + нормализация (среднее и std для MNIST)
# Здесь же выполняется приведение к диапазону [0, 1] (rescale)
transform = transforms.Compose([
    transforms.ToTensor(),                      # автоматически делит на 255 -> [0, 1]
    transforms.Normalize((0.1307,), (0.3081,))  # стандартная нормализация для MNIST
])

# Загрузка обучающей и проверочной выборок
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

print(f'Размер обучающей выборки: {len(train_dataset)}')
print(f'Размер тестовой выборки: {len(test_dataset)}')
```

```
100%|██████████| 9.91M/9.91M [00:01<00:00, 5.62MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 134kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.26MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.1MB/s]
Размер обучающей выборки: 60000
Размер тестовой выборки: 10000
```

Рисунок 1.2.2 – Результат загрузки датасета MNIST

1.2.3 Визуализация изображений из датасета.

Аналогично первому разделу, для ознакомления с содержимым датасета выводится по одному случайному изображению из каждого класса. Изображения извлекаются непосредственно из объекта train_dataset через атрибуты data и targets.

Листинг 1.2.1 – Вывод изображений каждого класса

```
# Просмотр изображений каждого класса для ознакомления с датасетом
fig, axs = plt.subplots(1, 10, figsize=(25, 3))
# Получаем все метки
all_labels = train_dataset.targets.numpy()
all_images = train_dataset.data.numpy()
for i in range(10):
    label_indexes = np.where(all_labels == i)[0]
    index = random.choice(label_indexes)
    img = all_images[index]
    axs[i].imshow(img, cmap='gray')
    axs[i].set_title(f'Класс: {i}')
plt.show()
```

```
In [3]: # Просмотр изображений каждого класса для ознакомления с датасетом
fig, axs = plt.subplots(1, 10, figsize=(25, 3))

# Получаем все метки
all_labels = train_dataset.targets.numpy()
all_images = train_dataset.data.numpy()

for i in range(10):
    label_indexes = np.where(all_labels == i)[0]
    index = random.choice(label_indexes)
    img = all_images[index]
    axs[i].imshow(img, cmap='gray')
    axs[i].set_title(f'Класс: {i}')

plt.show()
```

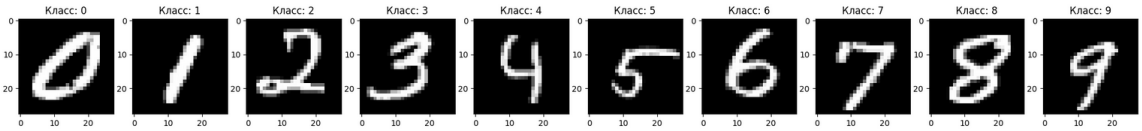


Рисунок 1.2.3 – Изображения из датасета MNIST

1.2.4 Создание объектов DataLoader.

В PyTorch для подачи данных в модель батчами используются объекты DataLoader. Они отвечают за разбиение датасета на батчи указанного размера, перемешивание данных (для обучающей выборки) и многопоточную загрузку. Размер батча установлен равным 128 в соответствии с заданием.

```
In [4]: # Создание DataLoader-ов с размером батча 128
BATCH_SIZE = 128

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

# Проверка размерностей
for images, labels in train_loader:
    print(f'Форма батча изображений: {images.shape}')   # [128, 1, 28, 28]
    print(f'Форма батча меток:       {labels.shape}')   # [128]
    break

Форма батча изображений: torch.Size([128, 1, 28, 28])
Форма батча меток:       torch.Size([128])
```

Рисунок 1.2.4 – Размерности батча в DataLoader

1.2.5 Создание модели сверточной нейронной сети.

В отличие от Keras, в PyTorch модель определяется как класс, наследник nn.Module. В методе __init__ объявляются все слои сети, в методе forward описывается порядок их применения к входным данным. Архитектура полностью повторяет Keras-версию: два сверточных блока (Conv2D → BatchNorm → MaxPooling → Dropout) и полносвязная часть с двумя слоями. Активация LeakyReLU применяется через функциональный интерфейс F.leaky_relu. После определения класса создаётся экземпляр модели и переносится на выбранное устройство. Также подсчитывается общее количество обучаемых параметров.

Листинг 1.2.2 – Класс модели CNN

```
class CNN(nn.Module):
    def __init__(self, num_classes=10):
        super(CNN, self).__init__()

        #  Первый сверточный блок
        # Conv2D: 1 -> 32 канала, ядро 3x3, padding=1 (аналог padding='same')
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
padding=1)
        self.bn1 = nn.BatchNorm2d(32)              # Слой нормализации
        self.pool1 = nn.MaxPool2d(2, 2)            # Слой подвыборки 2x2 -> 14x14
        self.dropout1 = nn.Dropout2d(0.25)         # Слой регуляризации

        #  Второй сверточный блок
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,
padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(2, 2)            # 14x14 -> 7x7
        self.dropout2 = nn.Dropout2d(0.25)

        #  Полносвязная часть
        # После двух пулингов: 64 канала * 7 * 7 = 3136 признаков
        self.fc1 = nn.Linear(64 * 7 * 7, 256)
        self.bn_fc = nn.BatchNorm1d(256)
        self.dropout_fc = nn.Dropout(0.5)

        # Выходной слой: 10 классов (без активации - CrossEntropyLoss
применит softmax внутри)
        self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
```

```
        # Первый блок: Conv -> LeakyReLU -> BatchNorm -> Pool -> Dropout
        x = self.conv1(x)
        x = F.leaky_relu(x, negative_slope=0.1)
        x = self.bn1(x)
        x = self.pool1(x)
        x = self.dropout1(x)

        # Второй блок
        x = self.conv2(x)
        x = F.leaky_relu(x, negative_slope=0.1)
        x = self.bn2(x)
        x = self.pool2(x)
        x = self.dropout2(x)

        # Преобразование в вектор (аналог Flatten)
        x = x.view(x.size(0), -1)

        # Полносвязный блок
        x = self.fc1(x)
        x = F.leaky_relu(x, negative_slope=0.1)
        x = self.bn_fc(x)
        x = self.dropout_fc(x)

        # Выходной слой
        x = self.fc2(x)
        return x

# Создание модели и перенос на устройство
model = CNN(num_classes=10).to(device)
print(model)
```

Подсчёт параметров

total_params = sum(p.numel() for p in model.parameters())

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'\nВсего параметров: {total_params:,}')

print(f'Обучаемых параметров: {trainable_params:,}')

```
# Создание модели и перенос на устройство
model = CNN(num_classes=10).to(device)
print(model)

# Подсчёт параметров
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'\nВсего параметров: {total_params:,}')
print(f'Обучаемых параметров: {trainable_params:,}')
```

```
CNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (dropout1): Dropout2d(p=0.25, inplace=False)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (dropout2): Dropout2d(p=0.25, inplace=False)
  (fc1): Linear(in_features=3136, out_features=256, bias=True)
  (bn_fc): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (dropout_fc): Dropout(p=0.5, inplace=False)
  (fc2): Linear(in_features=256, out_features=10, bias=True)
)

Всего параметров: 825,162
Обучаемых параметров: 825,162
```

Рисунок 1.2.5 – Структура модели и количество параметров

1.2.6 Обучение модели.

В отличие от Keras, в PyTorch цикл обучения пишется явно: для каждой эпохи и каждого батча выполняется прямой проход (forward), вычисление функции потерь, обратный проход (backward) и шаг оптимизатора. Такой подход даёт большую гибкость, но требует написания большего количества кода.

Функция потерь nn.CrossEntropyLoss совмещает softmax и negative log-likelihood, поэтому в выходном слое модели softmax не используется, а метки передаются как индексы классов (без one-hot кодирования). Оптимизатор – Adam с шагом обучения 0.001.

После фазы обучения на каждой эпохе следует фаза валидации. Перед ней модель переводится в режим инференса вызовом model.eval(), а градиенты отключаются через контекстный менеджер torch.no_grad() – это экономит память и ускоряет вычисления, поскольку обратный проход не нужен.

Листинг 1.2.3 – Цикл обучения

```
# Функция потерь - категориальная кроссэнтропия
# В PyTorch CrossEntropyLoss совмещает softmax + nll_loss, поэтому метки
передаются как индексы (не OHE)
criterion = nn.CrossEntropyLoss()

# Оптимизатор - Adam с шагом обучения 0.001
optimizer = optim.Adam(model.parameters(), lr=0.001)

EPOCHS = 15

# Списки для хранения метрик
history = {
    'train_loss': [], 'train_acc': [],
    'val_loss': [],   'val_acc': []
}

for epoch in range(EPOCHS):
    #  Фаза обучения
    model.train()
    train_loss_sum = 0.0
    train_correct = 0
    train_total = 0

    for batch_idx, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)
```

```
    # Обнуление градиентов
    optimizer.zero_grad()

    # Прямой проход
    outputs = model(images)
    loss = criterion(outputs, labels)

    # Обратный проход и шаг оптимизатора
    loss.backward()
    optimizer.step()

    # Накопление метрик
    train_loss_sum += loss.item() * images.size(0)
    _, predicted = outputs.max(1)
    train_total += labels.size(0)
    train_correct += predicted.eq(labels).sum().item()

train_loss = train_loss_sum / train_total
train_acc = train_correct / train_total
#  Фаза валидации
model.eval()
val_loss_sum = 0.0
val_correct = 0
val_total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
```

```
            loss = criterion(outputs, labels)

            val_loss_sum += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            val_total += labels.size(0)
            val_correct += predicted.eq(labels).sum().item()

    val_loss = val_loss_sum / val_total
    val_acc = val_correct / val_total

    # Сохранение в историю
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    # Вывод деталей обучения
    print(f'Эпоха {epoch+1:2d}/{EPOCHS} | '
          f'train_loss: {train_loss:.4f} | train_acc: {train_acc:.4f} | '
          f'val_loss: {val_loss:.4f} | val_acc: {val_acc:.4f}')

print('\nОбучение завершено!')
```

```
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            val_loss_sum += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            val_total += labels.size(0)
            val_correct += predicted.eq(labels).sum().item()

    val_loss = val_loss_sum / val_total
    val_acc = val_correct / val_total

    # Сохранение в историю
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    # Вывод деталей обучения
    print(f'Эпоха {epoch+1:2d}/{EPOCHS} | '
          f'train_loss: {train_loss:.4f} | train_acc: {train_acc:.4f} | '
          f'val_loss: {val_loss:.4f} | val_acc: {val_acc:.4f}')

print('\nОбучение завершено!')
```

```
Эпоха  1/15 | train_loss: 0.1530 | train_acc: 0.9545 | val_loss: 0.0460 | val_acc: 0.9853
Эпоха  2/15 | train_loss: 0.0695 | train_acc: 0.9782 | val_loss: 0.0303 | val_acc: 0.9900
Эпоха  3/15 | train_loss: 0.0546 | train_acc: 0.9829 | val_loss: 0.0292 | val_acc: 0.9904
Эпоха  4/15 | train_loss: 0.0476 | train_acc: 0.9845 | val_loss: 0.0254 | val_acc: 0.9918
Эпоха  5/15 | train_loss: 0.0397 | train_acc: 0.9877 | val_loss: 0.0238 | val_acc: 0.9922
Эпоха  6/15 | train_loss: 0.0353 | train_acc: 0.9892 | val_loss: 0.0255 | val_acc: 0.9917
Эпоха  7/15 | train_loss: 0.0337 | train_acc: 0.9893 | val_loss: 0.0222 | val_acc: 0.9922
Эпоха  8/15 | train_loss: 0.0307 | train_acc: 0.9905 | val_loss: 0.0257 | val_acc: 0.9911
Эпоха  9/15 | train_loss: 0.0293 | train_acc: 0.9904 | val_loss: 0.0233 | val_acc: 0.9924
Эпоха 10/15 | train_loss: 0.0282 | train_acc: 0.9907 | val_loss: 0.0208 | val_acc: 0.9937
Эпоха 11/15 | train_loss: 0.0234 | train_acc: 0.9922 | val_loss: 0.0216 | val_acc: 0.9927
Эпоха 12/15 | train_loss: 0.0255 | train_acc: 0.9917 | val_loss: 0.0236 | val_acc: 0.9934
Эпоха 13/15 | train_loss: 0.0211 | train_acc: 0.9931 | val_loss: 0.0202 | val_acc: 0.9932
Эпоха 14/15 | train_loss: 0.0212 | train_acc: 0.9929 | val_loss: 0.0237 | val_acc: 0.9925
Эпоха 15/15 | train_loss: 0.0207 | train_acc: 0.9934 | val_loss: 0.0225 | val_acc: 0.9925

Обучение завершено!
```

Рисунок 1.2.6 – Результат обучения модели по эпохам

Из выводимых метрик видно, что уже после первой эпохи модель достигает точности 98.53% на проверочной выборке. К пятнадцатой эпохе точность на обучающей выборке составляет 99.34%, на проверочной – 99.25%, что является высоким результатом и демонстрирует успешное обучение без переобучения.

1.2.7 Построение графика обучения.

Поскольку история обучения сохранялась в обычный словарь Python, графики строятся аналогично Keras-версии. Выводятся две диаграммы: динамика accuracy и динамика loss на обучающей и проверочной выборках.

Листинг 1.2.4 – Построение графика обучения

```
plt.figure(figsize=(12, 5))
```

```
# График accuracy
plt.subplot(1, 2, 1)
plt.plot(history['train_acc'], label='Обучающая выборка')
plt.plot(history['val_acc'], label='Проверочная выборка')
plt.title('Доля верных ответов (accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# График Loss
plt.subplot(1, 2, 2)
plt.plot(history['train_loss'], label='Обучающая выборка')
plt.plot(history['val_loss'], label='Проверочная выборка')
plt.title('Функция ошибки (loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print(f'Лучшая точность на проверочной выборке:
{max(history["val_acc"]):.4f} '
    f'({max(history["val_acc"])*100:.2f}%)')
```

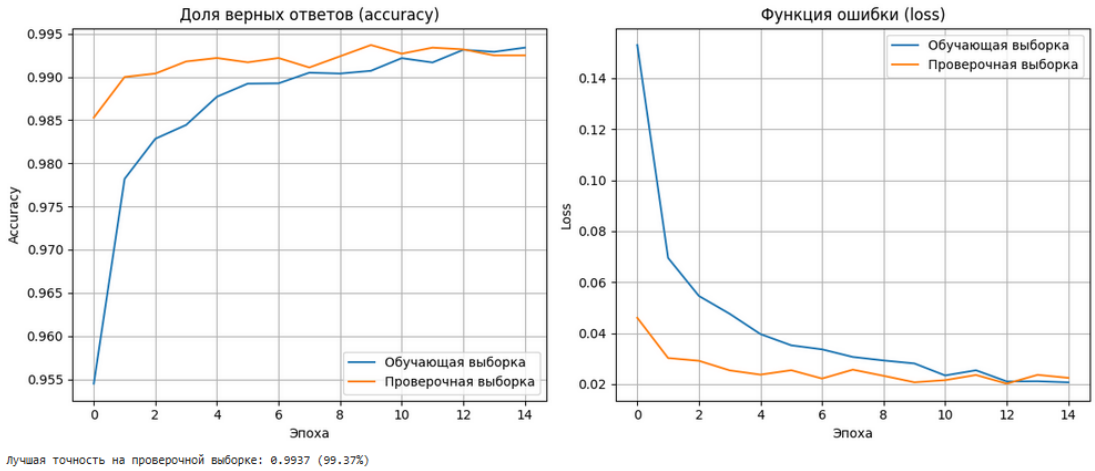


Рисунок 1.2.7 – График процесса обучения модели

Интересная особенность графика: точность на проверочной выборке на первых эпохах превышает точность на обучающей. Это объясняется работой слоёв Dropout и BatchNorm в режиме обучения – они вносят дополнительный шум, ухудшающий метрики на тренировочных данных, тогда как при валидации модель работает в детерминированном режиме (model.eval()), показывая истинное качество. К концу обучения обе кривые сходятся и стабилизируются около 99.3%, что подтверждает корректную работу модели.

1.2.8 Построение матрицы ошибок.

Для оценки качества классификации по каждому классу строится нормализованная матрица ошибок. Предсказания модели на тестовой выборке собираются в списки, которые затем передаются в функцию confusion_matrix из библиотеки scikit-learn.

Листинг 1.2.5 – Построение матрицы ошибок

```
# Получение всех предсказаний на тестовой выборке
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
```

```
        outputs = model(images)
        _, predicted = outputs.max(1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Построение нормализованной матрицы ошибок
cm = confusion_matrix(all_labels, all_preds, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=np.arange(10))
disp.plot(ax=ax, cmap='Blues', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

# Общая точность
accuracy = (all_preds == all_labels).mean()
print(f'\nОбщая точность на тестовой выборке: {accuracy:.4f}
({accuracy*100:.2f}%)')
```

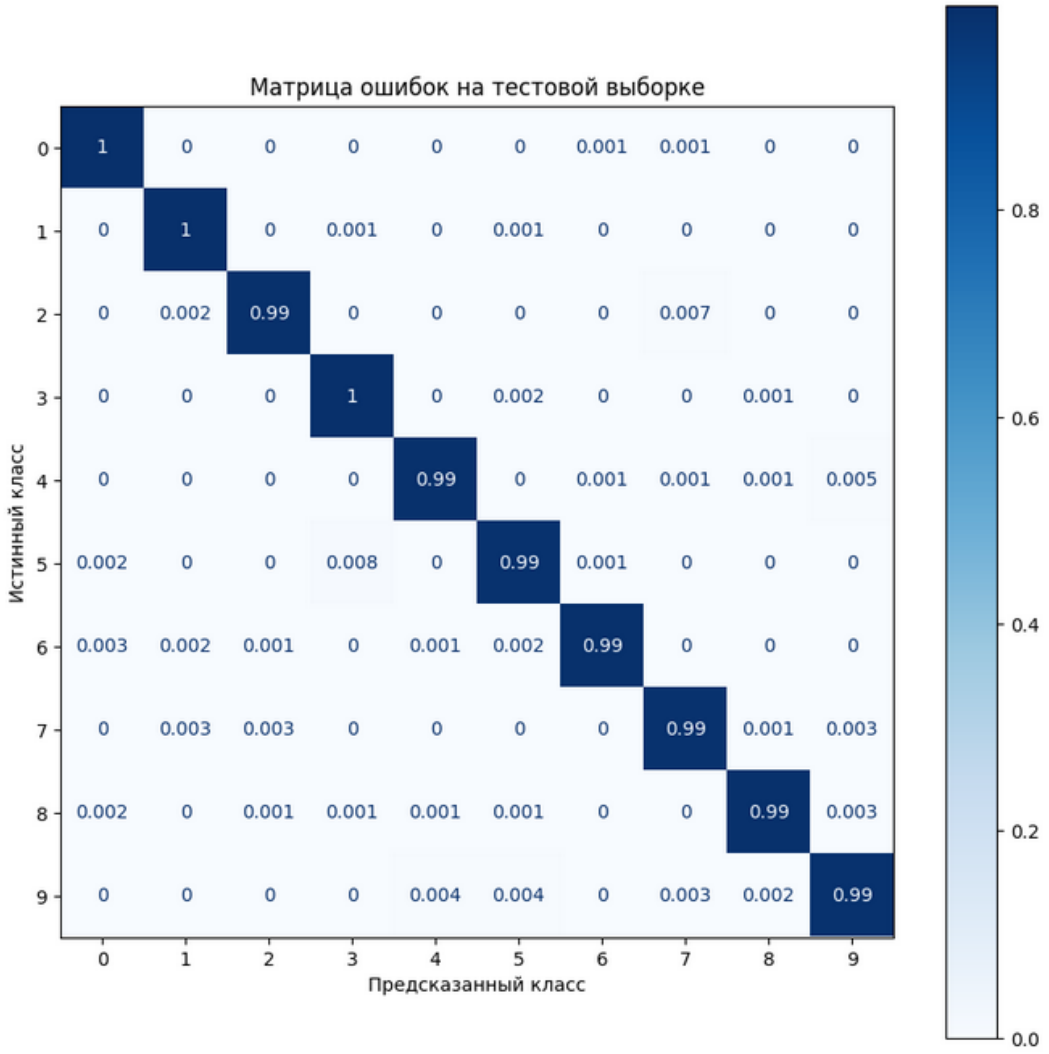


Рисунок 1.2.8 – Матрица ошибок на тестовой выборке

Анализ матрицы ошибок показывает, что модель PyTorch демонстрирует точность 99-100% по каждому классу. Цифры 0, 1 и 3 распознаются практически идеально (100%), для остальных классов точность составляет 99%. Общая точность распознавания на тестовой выборке – 99.25%, что превышает результат Keras-версии (98.79%).

Выводы: По результатам обучения обе модели показали высокую точность распознавания: Keras-версия – 98.79% на тестовой выборке, PyTorch-версия – 99.25%. Анализ графиков обучения и матриц ошибок подтвердил отсутствие переобучения и корректную работу обеих моделей на всех классах исходного датасета.

Сравнение фреймворков показало, что Keras обеспечивает более лаконичный код за счёт высокоуровневого API и встроенного метода fit(), тогда как PyTorch требует написания явного цикла обучения, но даёт большую гибкость в управлении процессом.

Задание 2. Решить задачу бинарной классификации изображений пассажиров автобуса по двум классам: «Входящий» и «Выходящий».

Цель – добиться точности работы модели выше 90% на проверочной выборке. Изображения представляют собой кадры с камер видеонаблюдения, на которых запечатлены пассажиры в полный рост в момент входа в автобус или выхода из него.

2.1 Подключение необходимых библиотек.

Для решения задачи импортируются модули для работы с изображениями (PIL, tensorflow.keras.preprocessing.image), массивами (numpy), графиками (matplotlib), файловой системой (os), генерации случайных чисел (random) и загрузки данных (gdown). Из библиотеки scikit-learn подключаются функции train_test_split для стратифицированного разбиения выборки, compute_class_weight для автоматического расчёта весов классов и confusion_matrix с classification_report для оценки качества модели. Из tensorflow.keras импортируются необходимые слои сети (Conv2D, MaxPooling2D, Dense, Dropout, BatchNormalization, LeakyReLU, GlobalAveragePooling2D), генератор аугментированных изображений ImageDataGenerator, колбэки EarlyStopping и ReduceLROnPlateau, а также оптимизатор Adam.

Классификация пассажиров автобуса (входящие / выходящие)

Цель: добиться точности работы модели выше 90% на проверочной выборке.

Загрузка данных

In [1]:

```python
# загрузка библиотек

# Методы для отрисовки изображений
from PIL import Image

# Для отрисовки графиков
import matplotlib.pyplot as plt

# Для генерации случайных чисел
import random

# Библиотека работы с массивами
import numpy as np

# Для работы с файлами
import os

# Импортируем модуль для загрузки данных
import gdown

# Для разделения выборок
from sklearn.model_selection import train_test_split

# Для автоматического расчёта весов классов
from sklearn.utils.class_weight import compute_class_weight

# Матрица ошибок и отчёт классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

# Генератор аугментированных изображений
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Для создания сети
from tensorflow.keras.models import Sequential

# Для создания слоёв
from tensorflow.keras.layers import (Dense, Conv2D, MaxPooling2D, Flatten,
                                     Dropout, BatchNormalization, LeakyReLU,
                                     GlobalAveragePooling2D)

# Для работы с изображениями
from tensorflow.keras.preprocessing import image

# Колбеки
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Преобразование меток в One-Hot
from tensorflow.keras.utils import to_categorical

# Оптимизатор
from tensorflow.keras.optimizers import Adam
```

Рисунок 2.1.1 – Подключение библиотек (общий вид)

Загрузка данных

In [1]:

```
# загрузка библиотек

# Методы для отрисовки изображений
from PIL import Image

# Для отрисовки графиков
import matplotlib.pyplot as plt

# Для генерации случайных чисел
import random

# Библиотека работы с массивами
import numpy as np

# Для работы с файлами
import os

# Импортируем модуль для загрузки данных
import gdown

# Для разделения выборок
from sklearn.model_selection import train_test_split

# Для автоматического расчёта весов классов
from sklearn.utils.class_weight import compute_class_weight

# Матрица ошибок и отчёт классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

# Генератор аугментированных изображений
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Для создания сети
from tensorflow.keras.models import Sequential

# Для создания слоёв
from tensorflow.keras.layers import (Dense, Conv2D, MaxPooling2D, Flatten,
                                     Dropout, BatchNormalization, LeakyReLU,
                                     GlobalAveragePooling2D)

# Для работы с изображениями
from tensorflow.keras.preprocessing import image

# Колбеки
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Преобразование меток в One-Hot
from tensorflow.keras.utils import to_categorical

# Оптимизатор
from tensorflow.keras.optimizers import Adam
```

Рисунок 2.1.2 – Подключение библиотек (детально)

2.2 Загрузка датасета.

Архив с изображениями загружается с облачного хранилища при помощи библиотеки gdown и распаковывается в директорию /content/bus. После распаковки выводится список папок-категорий, который содержит две директории, соответствующие классам задачи: «Входящий» и «Выходящий».

```
In [2]:
# Загрузка датасета (.zip-архив)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l4/bus.zip', None, quiet=True)

# Распаковка архива
!unzip -q "bus.zip" -d /content/bus

# Папка с папками картинок, рассортированных по категориям
IMAGE_PATH = '/content/bus'
os.listdir(IMAGE_PATH)

Out[2]: ['Входящий', 'Выходящий']
```

Рисунок 2.2.1 – Загрузка и распаковка датасета bus.zip

2.3 Определение списка классов.

Список классов формируется автоматически на основе названий папок в директории датасета и сортируется по алфавиту, что обеспечивает воспроизводимое сопоставление меток. В результате имеем два класса: «Входящий» (индекс 0) и «Выходящий» (индекс 1).

```
In [3]:
# Определение списка имён классов
CLASS_LIST = sorted(os.listdir(IMAGE_PATH))

# Определение количества классов
CLASS_COUNT = len(CLASS_LIST)

print(f'Количество классов: {CLASS_COUNT}, метки классов: {CLASS_LIST}')

Количество классов: 2, метки классов: ['Входящий', 'Выходящий']
```

Рисунок 2.3.1 – Список классов и их количество

2.4 Гиперпараметры.

Все ключевые параметры решения вынесены в отдельный блок для удобства настройки. Доля тестовой выборки – 10%, доля проверочной выборки от обучающего набора – 20%. Размер изображения для нейросети выбран 64×128 пикселей с соотношением сторон 2:1, что соответствует фигуре человека в полный рост на кадре. Изображения трёхканальные (RGB).

Параметры аугментации подобраны с учётом специфики задачи: пределы поворота ±15°, сдвиги по обеим осям ±10%, увеличение/уменьшение ±10%, изменение яркости в диапазоне (0.7, 1.3). Горизонтальное отражение отключено, поскольку направление движения пассажира принципиально важно для классификации – зеркально отражённое изображение «Входящего» может выглядеть как «Выходящий».

Для обучения установлено максимальное число эпох 50 (фактическое число определит EarlyStopping), размер батча – 32, оптимизатор – Adam с шагом обучения 0.0005.

Гиперпараметры

```
In [4]:
# Параметры разделения выборок
TEST_SPLIT          = 0.1                   # Доля тестовых данных в общем наборе
VAL_SPLIT           = 0.2                   # Доля проверочной выборки в обучающем наборе

# Параметры изображения (соотношение 2:1 — соответствует фигуре человека в полный рост)
IMG_WIDTH           = 64                    # Ширина изображения для нейросети
IMG_HEIGHT          = 128                   # Высота изображения для нейросети
IMG_CHANNELS        = 3                     # RGB

# Параметры аугментации
ROTATION_RANGE      = 15                    # Пределы поворота в градусах
WIDTH_SHIFT_RANGE   = 0.1                   # Пределы сдвига по горизонтали
HEIGHT_SHIFT_RANGE  = 0.1                   # Пределы сдвига по вертикали
ZOOM_RANGE          = 0.1                   # Пределы увеличения/уменьшения
BRIGHTNESS_RANGE    = (0.7, 1.3)            # Пределы изменения яркости
HORIZONTAL_FLIP     = False                 # Без флипа: направление движения важно

# Параметры обучения
EPOCHS              = 50                    # Максимальное число эпох (EarlyStopping остановит раньше)
BATCH_SIZE          = 32                    # Размер батча
OPTIMIZER           = Adam(learning_rate=0.0005)  # Оптимизатор
```

Рисунок 2.4.1 – Заданные гиперпараметры

2.5 Загрузка изображений.

Изображения последовательно загружаются из всех директорий классов с одновременным приведением к целевому размеру 128×64. Каждое изображение преобразуется в массив numpy типа float32, метки классов записываются в отдельный массив. По завершении загрузки выводится общая статистика: количество изображений, форма массива и распределение по классам.

Всего загружено 9081 изображение, форма массива (9081, 128, 64, 3). Распределение классов: 6485 изображений «Входящих» и 2596 изображений «Выходящих», что показывает существенный дисбаланс классов (примерно 71% против 29%) – этот факт необходимо учесть при обучении.

Загрузка изображений

In [5]:

```
images = []
labels = []

for i, class_name in enumerate(CLASS_LIST):
    class_path = os.path.join(IMAGE_PATH, class_name)
    for img_name in os.listdir(class_path):
        img = image.load_img(os.path.join(class_path, img_name),
                             target_size=(IMG_HEIGHT, IMG_WIDTH))
        images.append(image.img_to_array(img))
        labels.append(i)

images = np.array(images, dtype='float32')
labels = np.array(labels)

print(f'Всего изображений: {len(images)}')
print(f'Форма массива: {images.shape}')
print(f'Распределение классов: {np.bincount(labels)}')
```

```
Всего изображений: 9081
Форма массива: (9081, 128, 64, 3)
Распределение классов: [6485 2596]
```

Рисунок 2.5.1 – Загрузка изображений и статистика по выборке

2.6 Просмотр примеров изображений.

Для визуального ознакомления с содержимым датасета выводится по одному изображению из каждого класса. Это позволяет убедиться в корректности загрузки и оценить характер исходных кадров: качество, освещение, ракурс, типичные позы пассажиров.

```
In [6]:
# Просмотр по одному изображению из каждого класса
fig, axs = plt.subplots(1, CLASS_COUNT, figsize=(8, 5))
for i in range(CLASS_COUNT):
    idx = np.where(labels == i)[0][0]
    axs[i].imshow(images[idx].astype('uint8'))
    axs[i].set_title(CLASS_LIST[i])
    axs[i].axis('off')
plt.show()
```

Рисунок 2.6.1 – Примеры изображений каждого класса

2.7 Разделение на train / val / test со стратификацией.

Для корректной оценки качества модели данные разбиваются на три выборки в два этапа. Сначала от полного набора отделяется тестовая выборка размером 10%. Затем оставшиеся данные разбиваются на обучающую и проверочную выборки в соотношении 80% / 20%. Параметр stratify обеспечивает сохранение исходного распределения классов в каждой выборке, а random_state=42 гарантирует воспроизводимость разбиения. Метки классов преобразуются в формат One-Hot Encoding для использования с функцией ошибки категориальной кроссэнтропии.

В результате получены: обучающий набор – 6537 изображений, проверочный – 1635 изображений, тестовый – 909 изображений.

Разделение на train / val / test со стратификацией

In [7]:

```
# Сначала отделяем тестовую выборку (10% от всех данных)
x_train_full, x_test, y_train_full, y_test = train_test_split(
    images, labels,
    test_size=TEST_SPLIT,
    random_state=42,
    stratify=labels
)

# Затем делим оставшиеся данные на обучающую и проверочную (20% от обучающего набора)
x_train, x_val, y_train_raw, y_val_raw = train_test_split(
    x_train_full, y_train_full,
    test_size=VAL_SPLIT,
    random_state=42,
    stratify=y_train_full
)

# Преобразуем метки в One-Hot Encoding
y_train = to_categorical(y_train_raw, num_classes=CLASS_COUNT)
y_val = to_categorical(y_val_raw, num_classes=CLASS_COUNT)
y_test_ohe = to_categorical(y_test, num_classes=CLASS_COUNT)

print(f'Обучающий набор:    {x_train.shape}, метки: {y_train.shape}')
print(f'Проверочный набор:  {x_val.shape}, метки: {y_val.shape}')
print(f'Тестовый набор:     {x_test.shape}, метки: {y_test_ohe.shape}')
```

```
Обучающий набор:    (6537, 128, 64, 3), метки: (6537, 2)
Проверочный набор:  (1635, 128, 64, 3), метки: (1635, 2)
Тестовый набор:     (909, 128, 64, 3), метки: (909, 2)
```

Рисунок 2.7.1 – Разделение выборки на train, val и test

2.8 Проверка распределения классов.

Для проверки корректности стратификации выводится распределение классов в каждой из трёх выборок. Во всех трёх наборах сохраняется исходное соотношение: примерно 71.4% объектов класса «Входящий» и 28.6% объектов класса «Выходящий», что подтверждает успешную работу стратифицированного разбиения.

```
In [8]:
# Проверка распределения классов в каждой выборке
def print_balance(y_data, set_name):
    if len(y_data.shape) > 1:
        y_data = np.argmax(y_data, axis=1)
    unique, counts = np.unique(y_data, return_counts=True)
    print(f'Распределение в {set_name}:')
    for i, count in enumerate(counts):
        print(f'  Класс {CLASS_LIST[i]}: {count} ({count/len(y_data)*100:.1f}%)')

print_balance(y_train, 'обучающем наборе')
print_balance(y_val, 'проверочном наборе')
print_balance(y_test_ohe, 'тестовом наборе')
```

```
Распределение в обучающем наборе:
  Класс Входящий: 4668 (71.4%)
  Класс Выходящий: 1869 (28.6%)
Распределение в проверочном наборе:
  Класс Входящий: 1168 (71.4%)
  Класс Выходящий: 467 (28.6%)
Распределение в тестовом наборе:
  Класс Входящий: 649 (71.4%)
  Класс Выходящий: 260 (28.6%)
```

Рисунок 2.8.1 – Распределение классов в обучающем, проверочном и тестовом наборах

2.9 Расчёт весов классов.

Для компенсации дисбаланса классов рассчитываются их веса при помощи функции compute_class_weight из scikit-learn в режиме «balanced». Этот режим присваивает каждому классу вес, обратно пропорциональный его частоте: классу «Входящий» – 0.700, классу «Выходящий» – 1.749. Полученные веса передаются в метод fit модели через параметр class_weight, что заставляет функцию ошибки сильнее реагировать на ошибки классификации миноритарного класса.

```
In [9]:
# Автоматический расчёт весов классов для компенсации дисбаланса
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_raw),
    y=y_train_raw
)
class_weight_dict = dict(enumerate(class_weights))
print(f'Веса классов: {class_weight_dict}')
```

```
Веса классов: {0: np.float64(0.7001928020565553), 1: np.float64(1.7487961476725522)}
```

Аугментация изображений

Рисунок 2.9.1 – Расчёт весов классов

2.10 Аугментация изображений.

Для обучающей выборки создаётся генератор ImageDataGenerator, выполняющий нормализацию пикселей (rescale=1./255) и набор случайных преобразований: повороты, сдвиги по горизонтали и вертикали, увеличение/уменьшение, изменение яркости. Аугментация повышает разнообразие обучающих данных и улучшает обобщающую способность модели.

Для проверочной и тестовой выборок создаются отдельные генераторы, выполняющие только нормализацию без аугментации. Это необходимо, чтобы метрики качества рассчитывались на исходных изображениях и были сравнимы между эпохами.

Аугментация изображений

In [10]:

```
# Генератор для обучающей выборки — нормализация + аугментация
train_datagen = ImageDataGenerator(
    rescale=1./255.,
    rotation_range=ROTATION_RANGE,
    width_shift_range=WIDTH_SHIFT_RANGE,
    height_shift_range=HEIGHT_SHIFT_RANGE,
    zoom_range=ZOOM_RANGE,
    brightness_range=BRIGHTNESS_RANGE,
    horizontal_flip=HORIZONTAL_FLIP,
)

# Для проверочной и тестовой выборок только нормализация
val_datagen = ImageDataGenerator(rescale=1./255.)
test_datagen = ImageDataGenerator(rescale=1./255.)

# Создание генераторов
train_generator = train_datagen.flow(x_train, y_train, batch_size=BATCH_SIZE, shuffle=True)
validation_generator = val_datagen.flow(x_val, y_val, batch_size=BATCH_SIZE, shuffle=False)
test_generator = test_datagen.flow(x_test, y_test_ohe, batch_size=BATCH_SIZE, shuffle=False)
```

Рисунок 2.10.1 – Создание генераторов для аугментации и нормализации

2.11 Проверка работы аугментации.

Для визуального контроля работы генератора реализована функция show_batch, выводящая заданное число изображений из батча в виде сетки. Это позволяет убедиться, что аугментированные изображения остаются интерпретируемыми и не выходят за рамки реалистичных преобразований.

```
In [11]:
# Проверка работы аугментации
def show_batch(batch, img_range=range(10), figsize=(20, 8), columns=5):
    num_images = len(img_range)
    rows = (num_images + columns - 1) // columns
    fig, axes = plt.subplots(rows, columns, figsize=figsize)
    axes = axes.flatten()
    for i, img_idx in enumerate(img_range):
        img = batch[0][img_idx]
        axes[i].imshow(np.clip(img, 0, 1))
        class_label_idx = np.argmax(batch[1][img_idx])
        axes[i].set_title(CLASS_LIST[class_label_idx])
        axes[i].axis('off')
    for j in range(i + 1, len(axes)):
        axes[j].axis('off')
    plt.tight_layout()
    plt.show()

example_batch = next(train_generator)
show_batch(example_batch, img_range=range(10), columns=5)
```

Рисунок 2.11.1 – Код функции show_batch

На выводе видно, что преобразования действительно применяются: изображения слегка повёрнуты, сдвинуты, отличаются по яркости. При этом сохраняется ключевой признак – ориентация фигуры пассажира относительно поручня и направление движения, что важно для корректной работы классификатора.

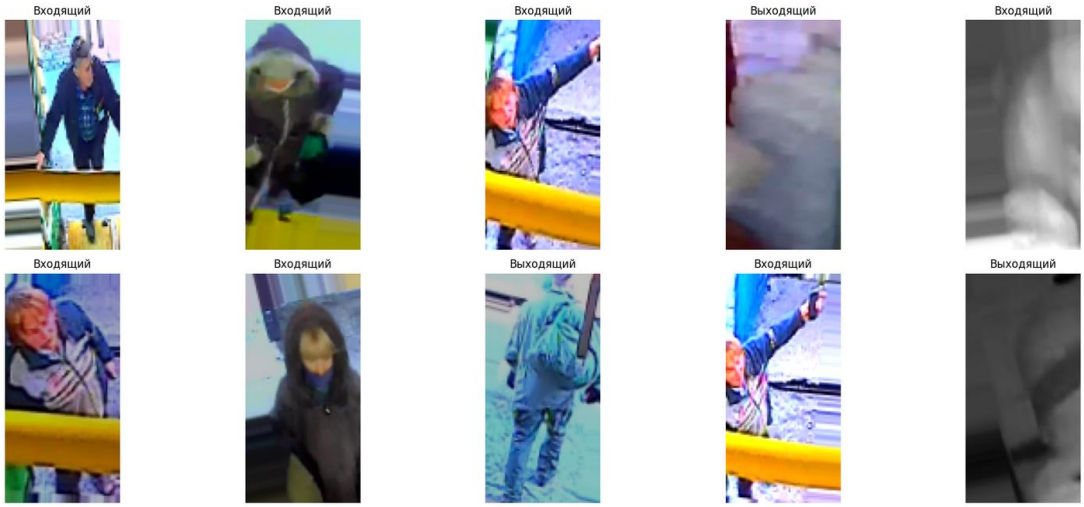


Рисунок 2.11.2 – Пример батча аугментированных изображений

2.12 Создание архитектуры модели.

Архитектура модели построена по паттерну VGG: три последовательных свёрточных блока с парными свёртками 3×3, нормализацией, активацией LeakyReLU и слоями подвыборки. Парные свёртки 3×3 эквивалентны одной свёртке 5×5 по рецептивному полю, но

требуют меньшего числа параметров и добавляют дополнительную нелинейность за счёт промежуточной активации.

Количество фильтров увеличивается от блока к блоку: 32 → 64 → 128, что соответствует общему принципу проектирования CNN – по мере уменьшения пространственных размеров карт признаков увеличивается их глубина. После каждого блока добавлен слой Dropout с возрастающей долей отключения нейронов (0.25 → 0.30 → 0.40), что повышает устойчивость к переобучению. Вместо традиционного слоя Flatten используется GlobalAveragePooling2D, который усредняет каждую карту признаков по пространственным координатам и резко сокращает число параметров полносвязной части. Полносвязный блок состоит из одного слоя Dense на 128 нейронов с нормализацией, активацией LeakyReLU и Dropout 0.5. Выходной слой – Dense на 2 нейрона (по числу классов) с активацией softmax.

Создание модели

Архитектура использует паттерн VGG: парные свёртки 3×3 в каждом блоке. Две свёртки 3×3 эквивалентны одной 5×5 по рецептивному полю, но требуют меньше параметров и добавляют дополнительную нелинейность.

In [12]:

```
cnn_model = Sequential([
    # ========== Блок 1: 32 фильтра ==========
    Conv2D(32, (3, 3), padding='same',
           input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    Conv2D(32, (3, 3), padding='same'),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    # ========== Блок 2: 64 фильтра ==========
    Conv2D(64, (3, 3), padding='same'),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    Conv2D(64, (3, 3), padding='same'),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.3),

    # ========== Блок 3: 128 фильтров ==========
    Conv2D(128, (3, 3), padding='same'),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    Conv2D(128, (3, 3), padding='same'),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.4),

    # ========== Полносвязная часть ==========
    GlobalAveragePooling2D(),  # резко сокращает параметры по сравнению с Flatten

    Dense(128),
    BatchNormalization(),
    LeakyReLU(alpha=0.1),
    Dropout(0.5),

    Dense(CLASS_COUNT, activation='softmax')
])

cnn_model.summary()
```

Рисунок 2.12.1 – Архитектура свёрточной нейронной сети

После создания модели вызывается метод summary(), который выводит сводку по слоям, формам выходных тензоров и количеству параметров. Общее число параметров модели – 306 082, из которых 304 930 обучаемых и 1 152 необучаемых (статистика BatchNormalization). Размер модели – около 1.17 МБ.

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:
he model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/activations/leaky_relu.py:41: Us
  warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 64, 32)	896
batch_normalization (BatchNormalization)	(None, 128, 64, 32)	128
leaky_re_lu (LeakyReLU)	(None, 128, 64, 32)	0
conv2d_1 (Conv2D)	(None, 128, 64, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 128, 64, 32)	128
leaky_re_lu_1 (LeakyReLU)	(None, 128, 64, 32)	0
max_pooling2d (MaxPooling2D)	(None, 64, 32, 32)	0
dropout (Dropout)	(None, 64, 32, 32)	0
conv2d_2 (Conv2D)	(None, 64, 32, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 64, 32, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 64, 32, 64)	0
conv2d_3 (Conv2D)	(None, 64, 32, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 64, 32, 64)	256
leaky_re_lu_3 (LeakyReLU)	(None, 64, 32, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 32, 16, 64)	0
dropout_1 (Dropout)	(None, 32, 16, 64)	0
conv2d_4 (Conv2D)	(None, 32, 16, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 32, 16, 128)	512
leaky_re_lu_4 (LeakyReLU)	(None, 32, 16, 128)	0
conv2d_5 (Conv2D)	(None, 32, 16, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 32, 16, 128)	512
leaky_re_lu_5 (LeakyReLU)	(None, 32, 16, 128)	0

Рисунок 2.12.2 – Структура модели (часть 1)

batch_normalization_3 (BatchNormalization)	(None, 64, 32, 64)	256
leaky_re_lu_3 (LeakyReLU)	(None, 64, 32, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 32, 16, 64)	0
dropout_1 (Dropout)	(None, 32, 16, 64)	0
conv2d_4 (Conv2D)	(None, 32, 16, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 32, 16, 128)	512
leaky_re_lu_4 (LeakyReLU)	(None, 32, 16, 128)	0
conv2d_5 (Conv2D)	(None, 32, 16, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 32, 16, 128)	512
leaky_re_lu_5 (LeakyReLU)	(None, 32, 16, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 16, 8, 128)	0
dropout_2 (Dropout)	(None, 16, 8, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
batch_normalization_6 (BatchNormalization)	(None, 128)	512
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

Total params: 306,082 (1.17 MB)

Trainable params: 304,930 (1.16 MB)

Non-trainable params: 1,152 (4.50 KB)

Рисунок 2.12.3 – Структура модели (часть 2) и общее количество параметров

2.13 Обучение модели.

Модель компилируется с функцией ошибки categorical_crossentropy, оптимизатором Adam с шагом обучения 0.0005 и метрикой accuracy. Для управления процессом обучения настроены два колбэка: EarlyStopping останавливает обучение при отсутствии улучшения val_accuracy в течение 7 эпох и восстанавливает веса лучшей эпохи; ReduceLROnPlateau снижает шаг обучения в 5 раз при стагнации val_loss в течение 3 эпох (минимальный шаг 1e-5). В метод fit передаётся словарь весов классов для компенсации дисбаланса.

Обучение модели

In [13]:

```
# Компиляция модели
cnn_model.compile(
    optimizer=OPTIMIZER,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Колбеки
early_stop = EarlyStopping(
    monitor='val_accuracy',
    patience=7,
    restore_best_weights=True,
    verbose=1
)

lr_reducer = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.2,
    patience=3,
    min_lr=1e-5,
    verbose=1
)

# Обучение
history = cnn_model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=[early_stop, lr_reducer],
    verbose=1
)
```

```
Epoch 1/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 52s 175ms/step - accuracy: 0.6101 - loss: 0.7554 - val_accuracy: 0.5896 - val_loss: 0.6389 - learning_rate: 5.0000e-04
Epoch 2/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.6885 - loss: 0.6035 - val_accuracy: 0.7486 - val_loss: 0.4540 - learning_rate: 5.0000e-04
Epoch 3/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 109ms/step - accuracy: 0.7271 - loss: 0.5378 - val_accuracy: 0.8177 - val_loss: 0.3925 - learning_rate: 5.0000e-04
Epoch 4/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 109ms/step - accuracy: 0.7326 - loss: 0.5113 - val_accuracy: 0.8239 - val_loss: 0.3605 - learning_rate: 5.0000e-04
Epoch 5/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.7669 - loss: 0.4606 - val_accuracy: 0.8098 - val_loss: 0.9688 - learning_rate: 5.0000e-04
Epoch 6/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 104ms/step - accuracy: 0.7733 - loss: 0.4461 - val_accuracy: 0.8294 - val_loss: 0.3870 - learning_rate: 5.0000e-04
Epoch 7/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 0s 102ms/step - accuracy: 0.7755 - loss: 0.4275
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.7764 - loss: 0.4374 - val_accuracy: 0.6783 - val_loss: 0.5780 - learning_rate: 5.0000e-04
Epoch 8/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 103ms/step - accuracy: 0.8042 - loss: 0.3800 - val_accuracy: 0.8502 - val_loss: 0.3156 - learning_rate: 1.0000e-04
Epoch 9/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8180 - loss: 0.3695 - val_accuracy: 0.8936 - val_loss: 0.2512 - learning_rate: 1.0000e-04
Epoch 10/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8273 - loss: 0.3608 - val_accuracy: 0.9095 - val_loss: 0.2322 - learning_rate: 1.0000e-04
Epoch 11/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8307 - loss: 0.3505 - val_accuracy: 0.8899 - val_loss: 0.2500 - learning_rate: 1.0000e-04
Epoch 12/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8297 - loss: 0.3385 - val_accuracy: 0.8550 - val_loss: 0.3278 - learning_rate: 1.0000e-04
```

Рисунок 2.13.1 – Код компиляции и запуска обучения, начало логов

В процессе обучения модель уверенно повышает точность на обучающей и проверочной выборках. Уже после первой эпохи val_accuracy составляет 0.5896, а к четвёртой эпохе модель преодолевает порог 80% (val_accuracy = 0.8239). Колбэк ReduceLROnPlateau сработал на 7-й эпохе,

снизив learning_rate до 1e-4, что привело к скачку val_accuracy до 0.8936 на 9-й эпохе. Дальнейшие снижения learning_rate (на 13-й и 22-й эпохах до 2e-5 и 1e-5 соответственно) обеспечили плавную сходимость.

```
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8400 - loss: 0.3319 - val_accuracy: 0.8813 - val_loss: 0.2581 - learning_rate: 1.0000e-04
Epoch 14/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8493 - loss: 0.3183 - val_accuracy: 0.9131 - val_loss: 0.2090 - learning_rate: 2.0000e-05
Epoch 15/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8533 - loss: 0.3093 - val_accuracy: 0.9119 - val_loss: 0.2112 - learning_rate: 2.0000e-05
Epoch 16/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 107ms/step - accuracy: 0.8484 - loss: 0.3122 - val_accuracy: 0.9138 - val_loss: 0.2039 - learning_rate: 2.0000e-05
Epoch 17/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 100ms/step - accuracy: 0.8541 - loss: 0.3045 - val_accuracy: 0.9168 - val_loss: 0.2018 - learning_rate: 2.0000e-05
Epoch 18/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 103ms/step - accuracy: 0.8548 - loss: 0.3103 - val_accuracy: 0.9205 - val_loss: 0.1992 - learning_rate: 2.0000e-05
Epoch 19/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8606 - loss: 0.3003 - val_accuracy: 0.9187 - val_loss: 0.1921 - learning_rate: 2.0000e-05
Epoch 20/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8573 - loss: 0.3121 - val_accuracy: 0.9187 - val_loss: 0.1974 - learning_rate: 2.0000e-05
Epoch 21/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8657 - loss: 0.2909 - val_accuracy: 0.9174 - val_loss: 0.2009 - learning_rate: 2.0000e-05
Epoch 22/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step - accuracy: 0.8647 - loss: 0.2999
Epoch 22: ReduceLROnPlateau reducing learning rate to 1e-05.
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8582 - loss: 0.2971 - val_accuracy: 0.9156 - val_loss: 0.1926 - learning_rate: 2.0000e-05
Epoch 23/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 96ms/step - accuracy: 0.8665 - loss: 0.2911 - val_accuracy: 0.9223 - val_loss: 0.1902 - learning_rate: 1.0000e-05
Epoch 24/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8663 - loss: 0.2911 - val_accuracy: 0.9229 - val_loss: 0.1839 - learning_rate: 1.0000e-05
Epoch 25/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8717 - loss: 0.2880 - val_accuracy: 0.9229 - val_loss: 0.1838 - learning_rate: 1.0000e-05
Epoch 26/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 103ms/step - accuracy: 0.8649 - loss: 0.2922 - val_accuracy: 0.9223 - val_loss: 0.1832 - learning_rate: 1.0000e-05
Epoch 27/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 104ms/step - accuracy: 0.8634 - loss: 0.2929 - val_accuracy: 0.9193 - val_loss: 0.1823 - learning_rate: 1.0000e-05
Epoch 28/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8603 - loss: 0.2909 - val_accuracy: 0.9254 - val_loss: 0.1840 - learning_rate: 1.0000e-05
Epoch 29/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8683 - loss: 0.2875 - val_accuracy: 0.9205 - val_loss: 0.1848 - learning_rate: 1.0000e-05
Epoch 30/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8613 - loss: 0.2918 - val_accuracy: 0.9217 - val_loss: 0.1828 - learning_rate: 1.0000e-05
Epoch 31/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8707 - loss: 0.2808 - val_accuracy: 0.9223 - val_loss: 0.1814 - learning_rate: 1.0000e-05
Epoch 32/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8672 - loss: 0.2842 - val_accuracy: 0.9248 - val_loss: 0.1816 - learning_rate: 1.0000e-05
Epoch 33/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8629 - loss: 0.2951 - val_accuracy: 0.9248 - val_loss: 0.1847 - learning_rate: 1.0000e-05
Epoch 34/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8634 - loss: 0.2871 - val_accuracy: 0.9291 - val_loss: 0.1766 - learning_rate: 1.0000e-05
Epoch 35/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 100ms/step - accuracy: 0.8697 - loss: 0.2867 - val_accuracy: 0.9272 - val_loss: 0.1739 - learning_rate: 1.0000e-05
Epoch 36/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8709 - loss: 0.2817 - val_accuracy: 0.9266 - val_loss: 0.1737 - learning_rate: 1.0000e-05
Epoch 37/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8707 - loss: 0.2788 - val_accuracy: 0.9205 - val_loss: 0.1873 - learning_rate: 1.0000e-05
Epoch 38/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8668 - loss: 0.2833 - val_accuracy: 0.9278 - val_loss: 0.1684 - learning_rate: 1.0000e-05
Epoch 39/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 104ms/step - accuracy: 0.8680 - loss: 0.2853 - val_accuracy: 0.9248 - val_loss: 0.1779 - learning_rate: 1.0000e-05
Epoch 40/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8657 - loss: 0.2849 - val_accuracy: 0.9229 - val_loss: 0.1839 - learning_rate: 1.0000e-05
Epoch 41/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8723 - loss: 0.2786 - val_accuracy: 0.9254 - val_loss: 0.1756 - learning_rate: 1.0000e-05
Epoch 41: early stopping
Restoring model weights from the end of the best epoch: 34.
```

Рисунок 2.13.2 – Логи обучения по эпохам

Обучение остановилось на 41-й эпохе по триггеру EarlyStopping, после чего были восстановлены веса лучшей 34-й эпохи. Лучшая точность на проверочной выборке составила 0.9291, что превышает целевое значение 90%, заданное в условии задачи.

2.14 Построение графика обучения.

Для визуальной оценки процесса обучения строятся два графика: динамика доли верных ответов и динамика функции ошибки на обучающей и проверочной выборках. На график accuracy дополнительно нанесена пунктирная линия целевой точности 90%. После построения графиков

выводится лучшее достигнутое значение val_accuracy и сообщение о достижении цели.

График обучения

```
In [14]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
         fig.suptitle('Процесс обучения модели')

         ax1.plot(history.history['accuracy'], label='Обучающий набор')
         ax1.plot(history.history['val_accuracy'], label='Проверочный набор')
         ax1.axhline(y=0.9, color='r', linestyle='--', label='Целевая точность 90%')
         ax1.set_xlabel('Эпоха обучения')
         ax1.set_ylabel('Доля верных ответов')
         ax1.legend()
         ax1.grid(True)

         ax2.plot(history.history['loss'], label='Обучающий набор')
         ax2.plot(history.history['val_loss'], label='Проверочный набор')
         ax2.set_xlabel('Эпоха обучения')
         ax2.set_ylabel('Ошибка')
         ax2.legend()
         ax2.grid(True)

         plt.tight_layout()
         plt.show()

         best_val_acc = max(history.history['val_accuracy'])
         print(f'\nЛучшая точность на проверочной выборке: {best_val_acc:.4f} ({best_val_acc*100:.2f}%)')
         if best_val_acc > 0.9:
             print('Цель достигнута! Точность выше 90%')
         else:
             print('Цель пока не достигнута')
```

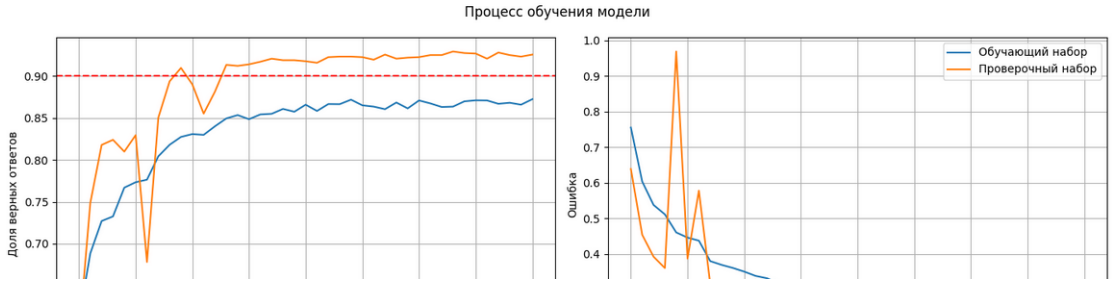
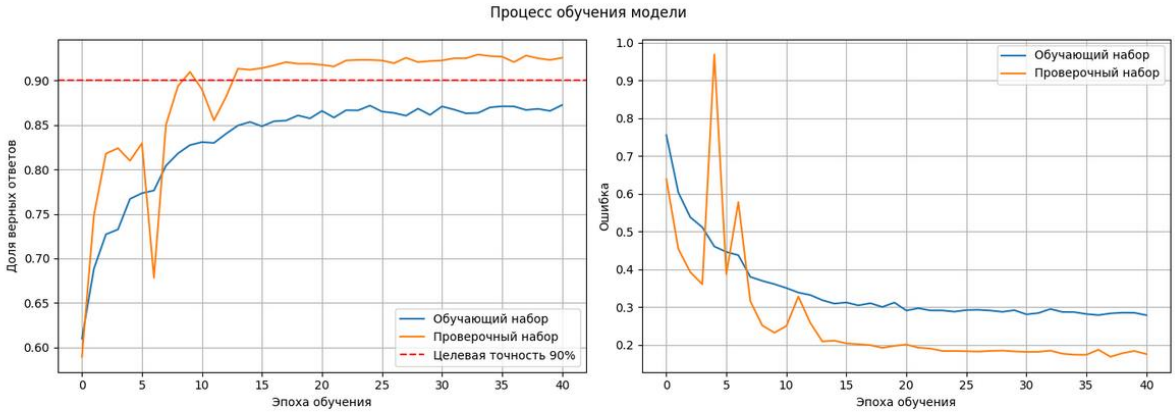


Рисунок 2.14.1 – Код построения графика обучения




```
Лучшая точность на проверочной выборке: 0.9291 (92.91%)
Цель достигнута! Точность выше 90%
```

Рисунок 2.14.2 – Графики процесса обучения модели

На графике accuracy видно, что точность на проверочной выборке быстро достигает целевого порога 90% (примерно к 9-й эпохе) и далее стабилизируется в диапазоне 91–93%. Точность на обучающей выборке растёт медленнее, что является следствием активной аугментации и регуляризации (Dropout, BatchNorm в режиме обучения), и к концу обучения находится в районе 86–87%. Такое соотношение, при котором val_accuracy выше train_accuracy, является нормальным для моделей с активным Dropout и

свидетельствует об отсутствии переобучения: модель не запоминает обучающую выборку, а действительно учится обобщать признаки. Лучшая точность на проверочной выборке составила 0.9291 (92.91%), что превышает целевое значение 90%.

График функции ошибки демонстрирует характерную картину: после резкого падения на первых эпохах кривая train_loss плавно снижается, а кривая val_loss сначала демонстрирует значительные колебания (что объясняется работой Dropout и аугментацией), но после снижения learning_rate выходит на плато значительно ниже обучающей ошибки – это также подтверждает отсутствие переобучения и эффективную работу регуляризации.

2.15 Оценка качества модели на тестовой выборке.

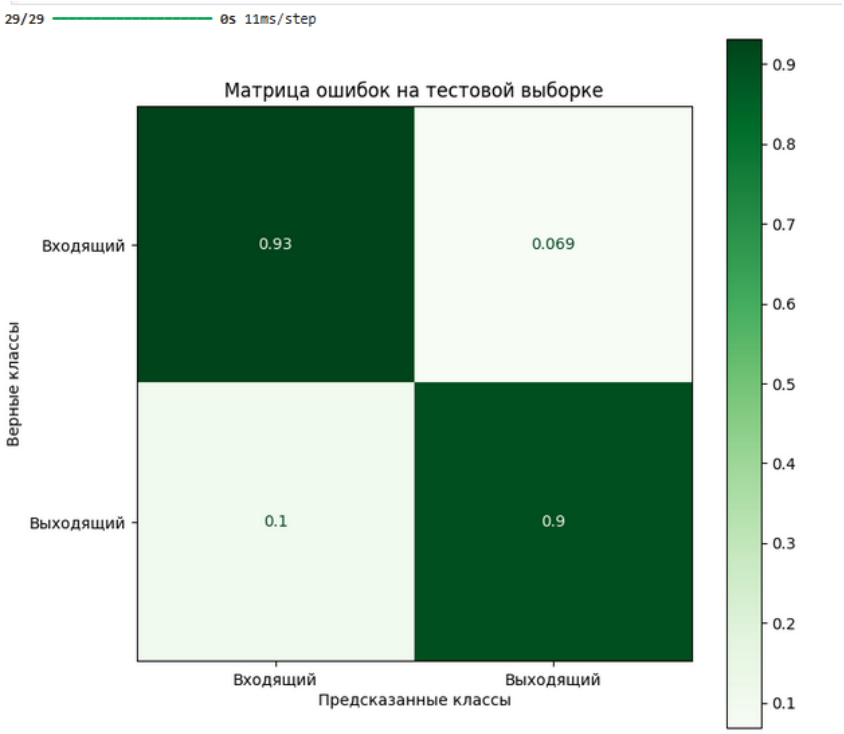
Для итоговой оценки модели используется тестовая выборка, не участвовавшая ни в обучении, ни в подборе гиперпараметров. Метод predict применяется ко всему тестовому генератору (test_generator.reset() гарантирует, что обход начнётся с первого элемента), результаты преобразуются из вероятностей в индексы классов через np.argmax. Затем строится нормализованная матрица ошибок и подробный отчёт по классам с метриками precision, recall и F1-score.

Оценка на тестовой выборке

In [18]:
```
# Получаем предсказания на ВСЕЙ тестовой выборке (а не на одном батче)
test_generator.reset()
y_pred = cnn_model.predict(test_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test  # исходные метки до OHE

# Матрица ошибок (нормализованная по строкам)
cm = confusion_matrix(y_true, y_pred_classes, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(8, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax, cmap='Greens', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанные классы')
plt.ylabel('Верные классы')
plt.show()

# Подробный отчёт: precision, recall, F1 по каждому классу
print('\nПодробный отчёт по классам:')
print(classification_report(y_true, y_pred_classes, target_names=CLASS_LIST))
```

```
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
```

Рисунок 2.15.1 – Код оценки модели на тестовой выборке



Подробный отчёт по классам:

	precision	recall	f1-score	support
Входящий	0.96	0.93	0.94	649
Выходящий	0.84	0.90	0.87	260
accuracy			0.92	909
macro avg	0.90	0.92	0.91	909
weighted avg	0.92	0.92	0.92	909

Рисунок 2.15.2 – Матрица ошибок и подробный отчёт по классам

Анализ матрицы ошибок и отчёта по классам показывает, что модель уверенно различает оба класса. Класс «Входящий» распознаётся с recall 0.93 и precision 0.96 (F1 = 0.94), класс «Выходящий» – с recall 0.90 и precision 0.84 (F1 = 0.87). Более низкая precision миноритарного класса – ожидаемое следствие компенсации дисбаланса через class_weight: модель «охотнее» относит сомнительные случаи к «Выходящим», что повышает их recall за счёт некоторого снижения precision. Общая accuracy на тестовой выборке составила 0.92, средневзвешенный F1-score – 0.92, что подтверждает успешное достижение цели работы.

2.16 Визуальный просмотр предсказаний.

Для качественной оценки работы модели выводится сетка из 10 случайных изображений тестовой выборки с указанием истинного и предсказанного класса, а также уверенности модели. Заголовки изображений

окрашены в зелёный цвет при правильном предсказании и в красный – при ошибке, что позволяет наглядно увидеть характер допускаемых ошибок.

```
In [16]:
# Визуальный просмотр предсказаний на случайных примерах
fig, axs = plt.subplots(2, 5, figsize=(20, 8))
axs = axs.flatten()

indices = random.sample(range(len(x_test)), 10)
for i, idx in enumerate(indices):
    img = (x_test[idx] / 255.0).clip(0, 1)
    true_label = CLASS_LIST[y_test[idx]]
    pred_label = CLASS_LIST[y_pred_classes[idx]]
    pred_proba = y_pred[idx][y_pred_classes[idx]]

    color = 'green' if true_label == pred_label else 'red'
    axs[i].imshow(img)
    axs[i].set_title(f'Истина: {true_label}\nПредсказано: {pred_label} ({pred_proba:.2f})',
                     color=color, fontsize=10)
    axs[i].axis('off')

plt.tight_layout()
plt.show()
```

Рисунок 2.16.1 – Код визуального просмотра предсказаний

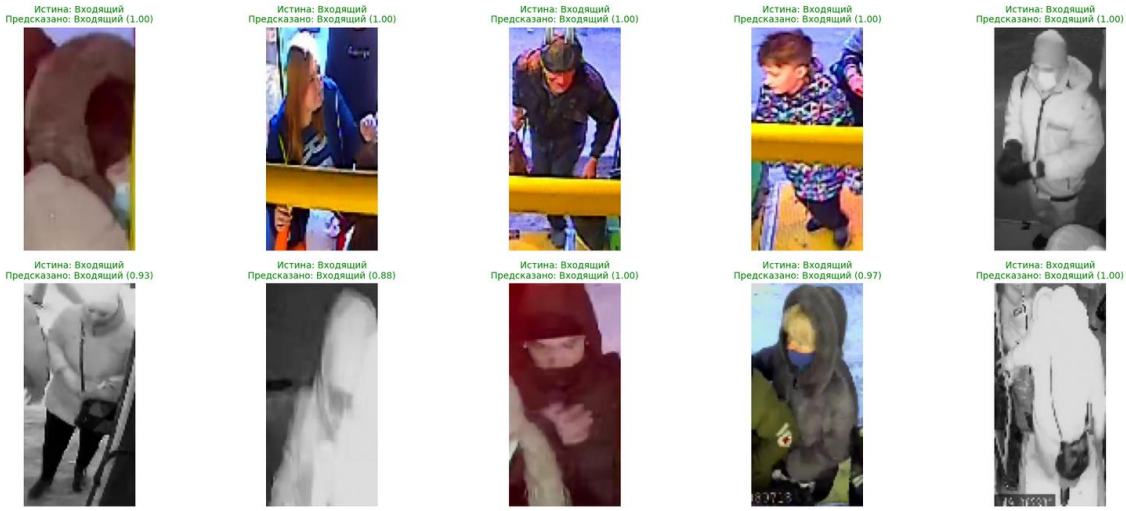


Рисунок 2.16.2 – Примеры предсказаний модели на случайных изображениях

На приведённой выборке все 10 изображений класса «Входящий» классифицированы правильно с уверенностью от 0.88 до 1.00. Видно, что модель уверенно работает на разных условиях съёмки: цветных и монохромных кадрах, разных ракурсах, при наличии частичных перекрытий поручнем. Это подтверждает, что аугментация данных и архитектура модели позволяют ей обобщать признаки направления движения пассажира независимо от внешних условий кадра.

2.17 Сохранение обученной модели.

После завершения обучения и оценки качества модель сохраняется на диск в современном формате Keras (.keras), который содержит архитектуру, веса и состояние оптимизатора в одном файле. Это позволяет в дальнейшем загрузить модель методом keras.models.load_model и использовать её для инференса без необходимости повторного обучения.

```
In [17]: # Сохранение модели
         cnn_model.save('cnn_model_pro_tf.keras')
         print('Модель сохранена в cnn_model_pro_tf.keras')

         Модель сохранена в cnn_model_pro_tf.keras
```

Рисунок 2.17.1 – Сохранение обученной модели в файл

Выводы: В рамках задания была решена задача бинарной классификации изображений пассажиров автобуса на классы «Входящий» и «Выходящий» с использованием свёрточной нейронной сети, построенной по паттерну VGG. Лучшая точность на проверочной выборке составила 92.91%, на тестовой – 92%, что превышает целевое значение 90%, заданное в условии.

Ключевыми факторами успеха стали: учёт специфики задачи (отключение горизонтального отражения, выбор соотношения сторон 2:1 для фигуры человека); компенсация дисбаланса классов (71% / 29%) через автоматический расчёт весов; использование аугментации для повышения обобщающей способности; применение GlobalAveragePooling2D вместо Flatten для сокращения числа параметров; динамическое управление шагом обучения через ReduceLROnPlateau и автоматическая остановка обучения через EarlyStopping с восстановлением лучших весов.

Задание 3. Решить задачу классификации автомобилей по маркам с применением технологии Transfer Learning.

Цель – добиться точности распознавания не ниже 93% на проверочной выборке. В качестве предобученной базы используется модель MobileNetV2, обученная на датасете ImageNet (~1.4 млн изображений). База данных задачи – middle_fmr.zip из ресурсов УИИ, содержащая изображения трёх марок: Ferrari, Mercedes, Renault.

Идея Transfer Learning состоит в том, чтобы взять модель, уже обученную решать общую задачу распознавания изображений, и дообучить её последние слои под конкретную задачу. Это позволяет получить высокую точность на относительно небольших датасетах за короткое время обучения, поскольку низкоуровневые признаки (границы, текстуры, простые формы) уже выучены базой и не требуют переобучения.

3.1 Подключение библиотек.

Помимо стандартных библиотек numpy, matplotlib, PIL, gdown и инструментов scikit-learn (train_test_split, compute_class_weight, метрики), импортируются модули TensorFlow / Keras для построения функциональной модели: Model из keras.models, слои Dense, Dropout, GlobalAveragePooling2D, BatchNormalization, Input, генератор аугментации ImageDataGenerator и колбэки EarlyStopping и ReduceLROnPlateau. Ключевыми для Transfer Learning являются импорты MobileNetV2 из tensorflow.keras.applications и функции preprocess_input из соответствующего модуля mobilenet_v2 – она выполняет нормализацию входных данных в формат, ожидаемый предобученной моделью.

Классификация автомобилей с использованием Transfer Learning

Цель: добиться точности распознавания не ниже 93% на проверочной выборке.

Подход: transfer learning на базе предобученной модели MobileNetV2 из `tf.keras.applications`.

База данных: middle_fmr.zip (база автомобилей УИИ).

Идея transfer learning: взять модель, уже обученную на ImageNet (~1.4 млн изображений), и дообучить её последние слои под нашу задачу. Это даёт высокую точность на маленьких датасетах за короткое время обучения.

Загрузка библиотек

[1]:
```
# Базовые библиотеки
import os
import random
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import gdown

# sklearn
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

# Keras / TensorFlow
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (Dense, Dropout, GlobalAveragePooling2D,
                                     BatchNormalization, Input)
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

# Предобученная архитектура
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
```

Рисунок 3.1.1 – Подключение библиотек и предобученной архитектуры

3.2 Загрузка датасета.

Архив с изображениями автомобилей (middle_fmr.zip) загружается с облачного хранилища УИИ при помощи gdown. После загрузки выполняется проверка размера файла: если архив подозрительно мал (меньше 1 МБ), выбрасывается AssertionError, что предохраняет от продолжения работы с повреждённым архивом. Затем архив распаковывается в директорию /content/cars и выводится список папок-классов: Ferrari, Mercedes, Renault.

Загрузка датасета

In [2]:

```
# Загрузка zip-архива из облака УИИ
DATASET_URL = 'https://storage.yandexcloud.net/aiueducation/Content/base/l5/middle_fmr.zip'
DATASET_ZIP = 'middle_fmr.zip'

gdown.download(DATASET_URL, DATASET_ZIP, quiet=False)

# Проверка, что файл скачался корректно
size_mb = os.path.getsize(DATASET_ZIP) / 1024 / 1024
print(f'\nРазмер скачанного архива: {size_mb:.1f} МБ')
assert size_mb > 1, 'Архив подозрительно маленький, проверьте URL!'

# Распаковка архива
!unzip -q -o "{DATASET_ZIP}" -d /content/cars

# Папка с папками-классами
IMAGE_PATH = '/content/cars'
print('\nСодержимое:', os.listdir(IMAGE_PATH))
```

```
Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/l5/middle_fmr.zip
To: /content/middle_fmr.zip
100%|██████████| 119M/119M [00:05<00:00, 20.4MB/s]
Размер скачанного архива: 113.5 МБ

Содержимое: ['Ferrari', 'Mercedes', 'Renault']
```

Рисунок 3.2.1 – Загрузка и распаковка датасета middle_fmr.zip

3.3 Определение списка классов.

При наличии вложенной структуры архива (если внутри корневой директории оказывается единственная папка) производится автоматическая корректировка пути к классам. Список классов формируется только из директорий (для устойчивости к посторонним файлам в корне), сортируется по алфавиту и подсчитывается размер каждого класса. Датасет содержит три класса: Ferrari (1088 фото), Mercedes (1161 фото), Renault (1178 фото). Распределение почти сбалансировано – соотношение классов близко к 1:1:1.

```
In [3]:
# Если внутри один уровень вложенности - корректируем путь
contents = os.listdir(IMAGE_PATH)
if len(contents) == 1 and os.path.isdir(os.path.join(IMAGE_PATH, contents[0])):
    IMAGE_PATH = os.path.join(IMAGE_PATH, contents[0])
    print(f'Путь скорректирован: {IMAGE_PATH}')

# Список классов
CLASS_LIST = sorted([d for d in os.listdir(IMAGE_PATH)
                     if os.path.isdir(os.path.join(IMAGE_PATH, d))])
CLASS_COUNT = len(CLASS_LIST)

print(f'\nКоличество классов: {CLASS_COUNT}')
print(f'Метки классов: {CLASS_LIST}')

# Размер каждого класса
print('\nРазмер каждого класса:')
for cls in CLASS_LIST:
    n = len(os.listdir(os.path.join(IMAGE_PATH, cls)))
    print(f'  {cls}: {n} фото')

Количество классов: 3
Метки классов: ['Ferrari', 'Mercedes', 'Renault']

Размер каждого класса:
  Ferrari: 1088 фото
  Mercedes: 1161 фото
  Renault: 1178 фото
```

Рисунок 3.3.1 – Список классов и количество изображений в каждом

3.4 Гиперпараметры.

Параметры разделения выборки те же, что и в Задании 2: 10% – тестовая выборка, 20% от обучающего набора – проверочная. Размер изображения выбран 224×224 пикселя – это стандартный входной размер для предобученной модели MobileNetV2, при котором она показывает максимальную точность на ImageNet.

Обучение разбито на два этапа: на первом этапе (10 эпох) обучается только новая «голова» модели – верхние полносвязные слои – при замороженной базе MobileNetV2; на втором этапе (20 эпох) выполняется fine-tuning – размораживается часть верхних слоёв базы и вся модель дообучается с очень малым learning rate. Размер батча – 32.

Гиперпараметры

In [4]:
```
# Параметры разделения
TEST_SPLIT      = 0.1
VAL_SPLIT       = 0.2

# Размер изображения - 224x224 стандарт для MobileNetV2
IMG_WIDTH       = 224
IMG_HEIGHT      = 224
IMG_CHANNELS    = 3

# Параметры обучения
BATCH_SIZE      = 32
EPOCHS_FROZEN   = 10    # Этап 1: обучаем только голову
EPOCHS_FINETUNE = 20    # Этап 2: fine-tuning
```

Рисунок 3.4.1 – Заданные гиперпараметры

3.5 Загрузка изображений.

Изображения последовательно загружаются из всех директорий классов с приведением к размеру 224×224. По сравнению с Заданием 2 добавлен блок try/except: если файл не удалось загрузить (например, из-за повреждения или нестандартного формата), выводится сообщение и файл пропускается, не прерывая работу всего скрипта. Всего успешно загружено 3427 изображений, форма массива (3427, 224, 224, 3), распределение классов: 1088 / 1161 / 1178.

Загрузка изображений

In [5]:
```
images = []
labels = []

for i, class_name in enumerate(CLASS_LIST):
    class_path = os.path.join(IMAGE_PATH, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        try:
            img = image.load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            images.append(image.img_to_array(img))
            labels.append(i)
        except Exception as e:
            print(f'Пропущен {img_path}: {e}')

images = np.array(images, dtype='float32')
labels = np.array(labels)

print(f'Всего изображений: {len(images)}')
print(f'Форма массива: {images.shape}')
print(f'Распределение классов: {np.bincount(labels)}')
```

```
Всего изображений: 3427
Форма массива: (3427, 224, 224, 3)
Распределение классов: [1088 1161 1178]
```

Рисунок 3.5.1 – Загрузка изображений и статистика по выборке

3.6 Просмотр примеров изображений.

Для ознакомления с содержимым датасета выводится по одному изображению из каждого класса. Видно, что в датасете присутствуют качественные студийные и рекламные снимки автомобилей в разных условиях освещения и ракурсах. Это существенно отличает датасет от Задания 2, где использовались кадры с камер видеонаблюдения.

```
In [6]: # Просмотр по одному изображению из каждого класса
        fig, axs = plt.subplots(1, CLASS_COUNT, figsize=(4*CLASS_COUNT, 4))
        if CLASS_COUNT == 1:
            axs = [axs]

        for i in range(CLASS_COUNT):
            idx = np.where(labels == i)[0][0]
            axs[i].imshow(images[idx].astype('uint8'))
            axs[i].set_title(CLASS_LIST[i])
            axs[i].axis('off')
        plt.tight_layout()
        plt.show()
```

Рисунок 3.6.1 – Примеры изображений каждого класса

3.7 Разделение на train / val / test со стратификацией.

Разделение производится по той же двухэтапной схеме со стратификацией, что и в Задании 2. В результате получены три выборки: обучающая – 2467 изображений, проверочная – 617, тестовая – 343. Все массивы имеют форму (n, 224, 224, 3), метки переведены в формат One-Hot Encoding.

Разделение на train / val / test со стратификацией

```
In [7]:
# Сначала отделяем тестовую выборку
x_train_full, x_test, y_train_full, y_test = train_test_split(
    images, labels,
    test_size=TEST_SPLIT,
    random_state=42,
    stratify=labels
)

# Затем делим на обучающую и проверочную
x_train, x_val, y_train_raw, y_val_raw = train_test_split(
    x_train_full, y_train_full,
    test_size=VAL_SPLIT,
    random_state=42,
    stratify=y_train_full
)

# OHE
y_train = to_categorical(y_train_raw, num_classes=CLASS_COUNT)
y_val = to_categorical(y_val_raw, num_classes=CLASS_COUNT)
y_test_ohe = to_categorical(y_test, num_classes=CLASS_COUNT)

print(f'Обучающий набор:    {x_train.shape}')
print(f'Проверочный набор:  {x_val.shape}')
print(f'Тестовый набор:     {x_test.shape}')

Обучающий набор:    (2467, 224, 224, 3)
Проверочный набор:  (617, 224, 224, 3)
Тестовый набор:     (343, 224, 224, 3)
```

Рисунок 3.7.1 – Разделение выборки на train, val и test

3.8 Расчёт весов классов.

Хотя распределение классов в исходных данных близко к равномерному (32% / 34% / 34%), для устойчивости работы compute_class_weight в режиме «balanced» применяется и здесь. Полученные веса: 1.060 для Ferrari, 0.984 для Mercedes, 0.970 для Renault – веса близки к 1, что подтверждает практическую сбалансированность датасета. Тем не менее их передача в model.fit гарантирует корректное взвешивание ошибок при минимальных колебаниях частот в батчах.

```
In [8]:
# Веса классов на случай дисбаланса
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_raw),
    y=y_train_raw
)
class_weight_dict = dict(enumerate(class_weights))
print(f'Веса классов: {class_weight_dict}')

Веса классов: {0: np.float64(1.0502341421881651), 1: np.float64(0.983652312599681), 2: np.float64(0.9697327044025157)}
```

Рисунок 3.8.1 – Расчёт весов классов

3.9 Аугментация и подготовка данных.

Важная особенность работы с предобученными моделями: вместо стандартной нормализации rescale=1./255 необходимо использовать функцию preprocess_input из tensorflow.keras.applications.mobilenet_v2. Эта функция

приводит данные к диапазону [-1, 1], в котором обучалась MobileNetV2 на ImageNet – без такого согласования предобученные веса работают некорректно.

Параметры аугментации скорректированы под специфику задачи: повороты ±15°, сдвиги ±10%, диапазон масштабирования увеличен до ±15%, диапазон яркости расширен до (0.8, 1.2), включено горизонтальное отражение (horizontal_flip=True) – в отличие от задачи о пассажирах, для классификации марки автомобиля зеркальное отражение не меняет класса. Параметр fill_mode='nearest' определяет стратегию заполнения пикселей при сдвигах и поворотах.

Аугментация и подготовка данных

Важно: для предобученных моделей вместо стандартной нормализации `1/255` используется `preprocess_input` из `tf.keras.applications.mobilenet_v2`. Эта функция приводит данные к диапазону [-1, 1], в котором обучалась MobileNetV2.

In [9]:
```
# Генератор для обучения - аугментация + нормализация под MobileNetV2
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.15,
    brightness_range=(0.8, 1.2),
    horizontal_flip=True,
    fill_mode='nearest'
)

# Для val и test - только нормализация
val_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
test_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

train_generator = train_datagen.flow(x_train, y_train, batch_size=BATCH_SIZE, shuffle=True)
validation_generator = val_datagen.flow(x_val, y_val, batch_size=BATCH_SIZE, shuffle=False)
test_generator = test_datagen.flow(x_test, y_test_ohe, batch_size=BATCH_SIZE, shuffle=False)
```

Рисунок 3.9.1 – Создание генераторов с preprocess_input и аугментацией

3.10 Создание модели на базе MobileNetV2.

Архитектура итоговой модели состоит из четырёх частей. Первая часть – предобученная база MobileNetV2 с параметром include_top=False (без верхнего classification-слоя ImageNet) и weights='imagenet' (загружаются предобученные веса). На этапе создания модели база замораживается (base_model.trainable = False) – её 2 260 800 параметров не будут обучаться на первом этапе.

Вторая часть – GlobalAveragePooling2D, преобразующий карту признаков размера (7, 7, 1280) в вектор длиной 1280. Третья часть – новая «голова»: BatchNormalization, Dropout(0.3), Dense(128, relu), BatchNormalization, Dropout(0.4). Четвёртая часть – выходной Dense с активацией softmax по числу классов (3). Модель собирается через функциональный API Keras, что необходимо для последующей разморозки части базовых слоёв.

Создание модели на базе MobileNetV2

Архитектура:

1. **MobileNetV2** — предобученная база, без classification-головы
2. **GlobalAveragePooling2D** — преобразует карту признаков в вектор
3. **Dense + Dropout** — новая голова под нашу задачу
4. **Выходной Dense с softmax** — по числу классов

In [10]:

```
# Загружаем MobileNetV2 без верхнего слоя
base_model = MobileNetV2(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS),
    include_top=False,
    weights='imagenet',
    pooling=None
)

# ЭТАП 1: замораживаем базу
base_model.trainable = False

# Собираем модель
inputs = Input(shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
outputs = Dense(CLASS_COUNT, activation='softmax')(x)

model = Model(inputs, outputs)

print(f'Всего параметров:       {model.count_params():,}')
print(f'Обучаемых параметров:   {sum(tf.size(w).numpy() for w in model.trainable_weights):,}')
print(f'Замороженных параметров: {sum(tf.size(w).numpy() for w in model.non_trainable_weights):,}')
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Всего параметров:       2,427,971
Обучаемых параметров:   167,171
Замороженных параметров: 2,260,800
```

Рисунок 3.10.1 – Создание модели на базе MobileNetV2

Общее количество параметров модели – 2 427 971, из которых обучаемых на первом этапе – всего 167 171 (только новая голова), замороженных – 2 260 800 (база MobileNetV2). Соотношение обучаемых к общему числу составляет около 7%, что является типичным для Transfer Learning и обеспечивает быстрое обучение головы.

3.11 Этап 1: обучение головы (база заморожена).

При замороженной базе можно использовать относительно высокий learning rate – обучаются только новые слои с случайной инициализацией. Параметры компиляции: оптимизатор Adam с шагом 0.001, функция ошибки categorical_crossentropy, метрика accuracy.

Для управления обучением настроены колбэки: EarlyStopping с patience=4 по val_accuracy и восстановлением лучших весов; ReduceLROnPlateau с factor=0.3, patience=2 и минимальным шагом 1e-5 по val_loss. Этап 1 рассчитан на 10 эпох.

Этап 1: обучение головы (база заморожена)

С замороженной базой можем использовать высокий learning rate — обучаются только новые слои.

In [11]:

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

callbacks_stage1 = [
    EarlyStopping(monitor='val_accuracy', patience=4, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=2, min_lr=1e-5, verbose=1)
]

history_stage1 = model.fit(
    train_generator,
    epochs=EPOCHS_FROZEN,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=callbacks_stage1,
    verbose=1
)
```

```
Epoch 1/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 91s 890ms/step - accuracy: 0.6627 - loss: 0.9128 - val_accuracy: 0.8023 - val_loss: 0.4684 - learning_rate: 0.0010
Epoch 2/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 36s 459ms/step - accuracy: 0.7685 - loss: 0.6158 - val_accuracy: 0.8006 - val_loss: 0.4390 - learning_rate: 0.0010
Epoch 3/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 34s 435ms/step - accuracy: 0.7831 - loss: 0.5517 - val_accuracy: 0.8136 - val_loss: 0.4393 - learning_rate: 0.0010
Epoch 4/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 406ms/step - accuracy: 0.8058 - loss: 0.4939 - val_accuracy: 0.8233 - val_loss: 0.4135 - learning_rate: 0.0010
Epoch 5/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 424ms/step - accuracy: 0.8018 - loss: 0.4843 - val_accuracy: 0.8088 - val_loss: 0.4484 - learning_rate: 0.0010
Epoch 6/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 389ms/step - accuracy: 0.8077 - loss: 0.4736
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 400ms/step - accuracy: 0.8156 - loss: 0.4460 - val_accuracy: 0.8282 - val_loss: 0.4272 - learning_rate: 0.0010
Epoch 7/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 419ms/step - accuracy: 0.8285 - loss: 0.4079 - val_accuracy: 0.8282 - val_loss: 0.4213 - learning_rate: 3.0000e-04
Epoch 8/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 387ms/step - accuracy: 0.8335 - loss: 0.4187
Epoch 8: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 397ms/step - accuracy: 0.8322 - loss: 0.4129 - val_accuracy: 0.8314 - val_loss: 0.4161 - learning_rate: 3.0000e-04
Epoch 9/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 413ms/step - accuracy: 0.8253 - loss: 0.4176 - val_accuracy: 0.8331 - val_loss: 0.4087 - learning_rate: 9.0000e-05
Epoch 10/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 394ms/step - accuracy: 0.8379 - loss: 0.3887 - val_accuracy: 0.8314 - val_loss: 0.4061 - learning_rate: 9.0000e-05
Restoring model weights from the end of the best epoch: 9.
```

Рисунок 3.11.1 – Код компиляции, обучение головы и логи этапа 1

Уже после первой эпохи val_accuracy достигает 0.8023 – это демонстрирует мощь Transfer Learning: благодаря качественным предобученным признакам новая голова быстро находит разделяющую гиперплоскость в пространстве признаков. Колбэк ReduceLROnPlateau сработал на 6-й и 8-й эпохах, последовательно понижая learning rate до 3e-4 и 9e-5. Этап 1 завершился восстановлением лучших весов 9-й эпохи (val_accuracy ≈ 0.83).

3.12 Этап 2: Fine-tuning (разморозка верхних слоёв базы).

Идея fine-tuning состоит в том, чтобы дообучить высокоуровневые слои предобученной базы под специфику нашей задачи, не разрушая полезные низкоуровневые признаки. Для этого размораживается вся база (base_model.trainable = True), но нижние слои затем явно замораживаются обратно: оставляются обучаемыми только верхние ~30% слоёв (с индекса 100 и выше).

Критически важно перекомпилировать модель после изменения trainable-флагов слоёв и использовать очень малый learning rate (1e-5) – это позволяет деликатно адаптировать признаки, не разрушая их шумными большими шагами градиента. После перекомпиляции число обучаемых параметров возрастает до 2 028 611 – почти на порядок больше, чем на этапе 1.

Этап 2: Fine-tuning (размораживаем верхние слои базы)

Размораживаем верхнюю часть MobileNetV2 и дообучаем всю модель с очень маленьким learning rate. Это позволяет модели адаптировать высокоуровневые признаки под автомобили, не разрушая полезные низкоуровневые признаки.

In [12]:

```
# Размораживаем базу
base_model.trainable = True

# Замораживаем нижние слои - учим только верхние ~30%
fine_tune_at = 100
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Перекомпиляция с очень маленьким learning rate
model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

trainable_params = sum(tf.size(w).numpy() for w in model.trainable_weights)
print(f'Обучаемых параметров на этапе 2: {trainable_params:,}')
```

```
Обучаемых параметров на этапе 2: 2,028,611
```

Рисунок 3.12.1 – Разморозка верхних слоёв и перекомпиляция модели

In [13]:

```
callbacks_stage2 = [
    EarlyStopping(monitor='val_accuracy', patience=6, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3, min_lr=1e-7, verbose=1)
]

history_stage2 = model.fit(
    train_generator,
    epochs=EPOCHS_FINETUNE,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=callbacks_stage2,
    verbose=1
)
```

```
Epoch 1/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 78s 689ms/step - accuracy: 0.7471 - loss: 0.6327 - val_accuracy: 0.8282 - val_loss: 0.3980 - learning_rate: 1.0000e-05
Epoch 2/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 405ms/step - accuracy: 0.7783 - loss: 0.5441 - val_accuracy: 0.8104 - val_loss: 0.4581 - learning_rate: 1.0000e-05
Epoch 3/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8139 - loss: 0.4750 - val_accuracy: 0.8055 - val_loss: 0.4909 - learning_rate: 1.0000e-05
Epoch 4/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 514ms/step - accuracy: 0.8086 - loss: 0.4722
Epoch 4: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.
78/78 ━━━━━━━━━━━━━━━━━━━━ 41s 531ms/step - accuracy: 0.8079 - loss: 0.4767 - val_accuracy: 0.8169 - val_loss: 0.4760 - learning_rate: 1.0000e-05
Epoch 5/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 34s 427ms/step - accuracy: 0.8204 - loss: 0.4483 - val_accuracy: 0.8201 - val_loss: 0.4580 - learning_rate: 3.0000e-06
Epoch 6/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 404ms/step - accuracy: 0.8216 - loss: 0.4646 - val_accuracy: 0.8201 - val_loss: 0.4315 - learning_rate: 3.0000e-06
Epoch 7/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 413ms/step - accuracy: 0.8064 - loss: 0.4753
Epoch 7: ReduceLROnPlateau reducing learning rate to 8.999999636216671e-07.
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8200 - loss: 0.4572 - val_accuracy: 0.8298 - val_loss: 0.4146 - learning_rate: 3.0000e-06
Epoch 8/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 404ms/step - accuracy: 0.8273 - loss: 0.4266 - val_accuracy: 0.8363 - val_loss: 0.4050 - learning_rate: 9.0000e-07
Epoch 9/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8212 - loss: 0.4369 - val_accuracy: 0.8444 - val_loss: 0.3913 - learning_rate: 9.0000e-07
Epoch 10/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 410ms/step - accuracy: 0.8314 - loss: 0.4091 - val_accuracy: 0.8509 - val_loss: 0.3823 - learning_rate: 9.0000e-07
Epoch 11/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 421ms/step - accuracy: 0.8306 - loss: 0.4418 - val_accuracy: 0.8525 - val_loss: 0.3743 - learning_rate: 9.0000e-07
Epoch 12/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 417ms/step - accuracy: 0.8245 - loss: 0.4560 - val_accuracy: 0.8525 - val_loss: 0.3669 - learning_rate: 9.0000e-07
Epoch 13/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 411ms/step - accuracy: 0.8379 - loss: 0.4055 - val_accuracy: 0.8590 - val_loss: 0.3630 - learning_rate: 9.0000e-07
Epoch 14/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 428ms/step - accuracy: 0.8229 - loss: 0.4372 - val_accuracy: 0.8541 - val_loss: 0.3573 - learning_rate: 9.0000e-07
Epoch 15/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 426ms/step - accuracy: 0.8456 - loss: 0.3968 - val_accuracy: 0.8525 - val_loss: 0.3537 - learning_rate: 9.0000e-07
Epoch 16/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 411ms/step - accuracy: 0.8419 - loss: 0.4126 - val_accuracy: 0.8525 - val_loss: 0.3509 - learning_rate: 9.0000e-07
Epoch 17/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 428ms/step - accuracy: 0.8362 - loss: 0.4043 - val_accuracy: 0.8509 - val_loss: 0.3494 - learning_rate: 9.0000e-07
Epoch 18/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 409ms/step - accuracy: 0.8443 - loss: 0.3997 - val_accuracy: 0.8476 - val_loss: 0.3493 - learning_rate: 9.0000e-07
Epoch 19/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 420ms/step - accuracy: 0.8293 - loss: 0.4189 - val_accuracy: 0.8444 - val_loss: 0.3488 - learning_rate: 9.0000e-07
Epoch 19: early stopping
Restoring model weights from the end of the best epoch: 13.
```

Рисунок 3.12.2 – Логи обучения на этапе fine-tuning

На этапе 2 настроены более терпеливые колбэки: EarlyStopping с patience=6 и ReduceLROnPlateau с patience=3 и min_lr=1e-7. Из логов видно, что точность на проверочной выборке плавно растёт от 0.8282 (1-я эпоха) до 0.859 (13-я эпоха). EarlyStopping сработал на 19-й эпохе и восстановил веса лучшей 13-й. ReduceLROnPlateau последовательно снижал learning rate (4-я эпоха – до 3e-6, 7-я эпоха – до 9e-7), что обеспечило плавную сходимость без срывов.

3.13 Построение графика обучения.

Поскольку обучение разбито на два этапа, история сохранена в двух разных объектах history_stage1 и history_stage2. Для построения единого графика их метрики последовательно объединяются (списки accuracy, val_accuracy, loss, val_loss из обоих этапов). На графиках дополнительно отображаются: горизонтальная пунктирная линия целевой точности 93% и вертикальная линия в момент перехода с этапа 1 на этап 2 (на границе stage1_end - 0.5), что позволяет визуально оценить эффект fine-tuning.

График обучения

In [14]:

```
# Объединяем историю обоих этапов
acc = history_stage1.history['accuracy'] + history_stage2.history['accuracy']
val_acc = history_stage1.history['val_accuracy'] + history_stage2.history['val_accuracy']
loss = history_stage1.history['loss'] + history_stage2.history['loss']
val_loss = history_stage1.history['val_loss'] + history_stage2.history['val_loss']

stage1_end = len(history_stage1.history['accuracy'])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Процесс обучения')

ax1.plot(acc, label='Обучающий набор')
ax1.plot(val_acc, label='Проверочный набор')
ax1.axhline(y=0.93, color='r', linestyle='--', alpha=0.7, label='Целевая точность 93%')
ax1.axvline(x=stage1_end - 0.5, color='gray', linestyle=':', alpha=0.7, label='Начало fine-tuning')
ax1.set_xlabel('Эпоха')
ax1.set_ylabel('Доля верных ответов')
ax1.legend()
ax1.grid(True)

ax2.plot(loss, label='Обучающий набор')
ax2.plot(val_loss, label='Проверочный набор')
ax2.axvline(x=stage1_end - 0.5, color='gray', linestyle=':', alpha=0.7, label='Начало fine-tuning')
ax2.set_xlabel('Эпоха')
ax2.set_ylabel('Ошибка')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

best_val_acc = max(val_acc)
print(f'\nЛучшая точность на проверочной выборке: {best_val_acc:.4f} ({best_val_acc*100:.2f}%)')
if best_val_acc >= 0.93:
    print('Цель достигнута! Точность не ниже 93%')
else:
    print('Цель пока не достигнута')
```

Рисунок 3.13.1 – Код построения объединённого графика обучения

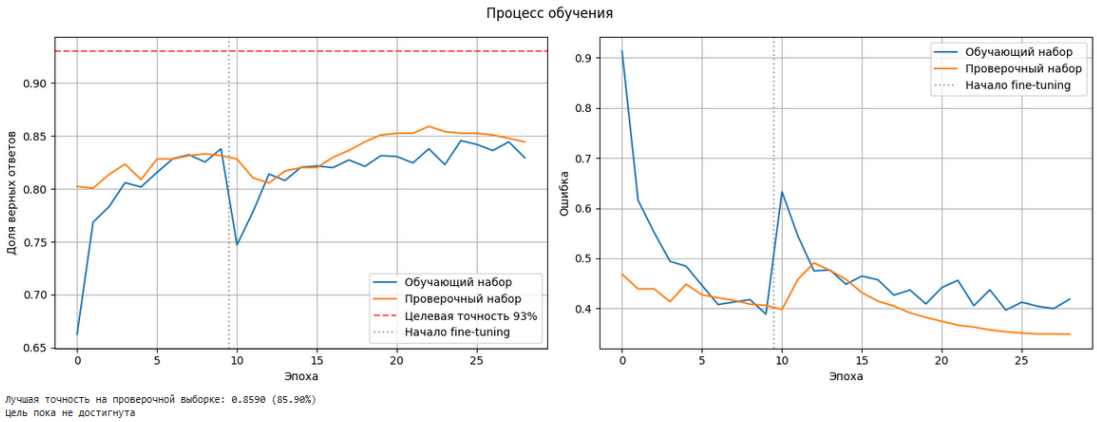


Рисунок 3.13.2 – Графики процесса обучения по обоим этапам

На графике accuracy чётко видны два этапа обучения, разделённые серой пунктирной линией. На первом этапе (0–9 эпохи, база заморожена) val_accuracy быстро поднимается с 0.80 до плато около 0.83 – это эффект Transfer Learning, когда новая голова за несколько эпох настраивается под уже выученные базой признаки. В момент перехода на второй этап (fine-tuning) видна резкая просадка train_accuracy до 0.74 и скачок train_loss – это типичное поведение при разморозке слоёв базы и пересчёте оптимизатора с нуля на очень малом learning rate. После 12-й эпохи модель плавно восстанавливается и продолжает медленно улучшаться: val_accuracy постепенно возрастает с 0.82 до 0.86.

Лучшая точность на проверочной выборке – 0.8590 (85.90%), что НЕ достигает целевого порога 93%. Возможные причины: сложность визуального различения некоторых ракурсов (например, кроссоверов Mercedes и Renault со схожими формами кузова), относительно небольшой объём обучающей выборки (2467 изображений на 3 класса), а также возможная избыточная регуляризация (Dropout 0.3 + 0.4) и слишком консервативный learning rate на этапе fine-tuning. Для достижения целевой точности можно было бы увеличить число эпох fine-tuning, ослабить Dropout, попробовать более глубокую базу (MobileNetV2-Large, EfficientNetB0) или применить дискриминативный learning rate.

3.14 Оценка качества модели на тестовой выборке.

Для итоговой оценки модели применяется тот же алгоритм, что и в Задании 2: метод predict на полном тестовом генераторе после reset(), преобразование вероятностей в индексы классов и построение нормализованной матрицы ошибок и подробного classification_report. Поскольку число классов больше двух, в matplotlib дополнительно установлена ротация подписей по оси X (xticks_rotation=45) для лучшей читаемости.

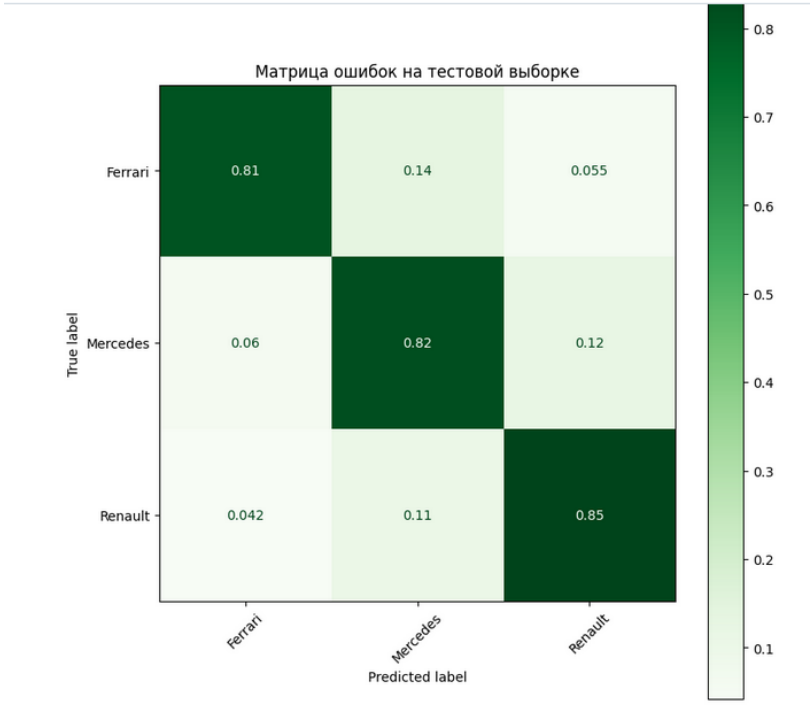
Оценка на тестовой выборке

In [18]:

```
test_generator.reset()
y_pred = model.predict(test_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test

# Матрица ошибок
cm = confusion_matrix(y_true, y_pred_classes, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(max(8, CLASS_COUNT), max(8, CLASS_COUNT)))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax, cmap='Greens', colorbar=True, xticks_rotation=45)
plt.title('Матрица ошибок на тестовой выборке')
plt.tight_layout()
plt.show()

print('\nПодробный отчёт по классам:')
print(classification_report(y_true, y_pred_classes, target_names=CLASS_LIST))
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
```

Рисунок 3.14.1 – Код оценки модели на тестовой выборке




```
Подробный отчёт по классам:
              precision    recall  f1-score   support

     Ferrari       0.88      0.81      0.84       109
    Mercedes       0.77      0.82      0.79       116
     Renault       0.83      0.85      0.84       118

    accuracy                           0.83       343
   macro avg       0.83      0.82      0.83       343
weighted avg       0.83      0.83      0.83       343
```

Рисунок 3.14.2 – Матрица ошибок и подробный отчёт по классам

Анализ матрицы ошибок показывает, что модель уверенно распознаёт все три класса с точностью на диагонали 0.81 / 0.82 / 0.85 для Ferrari, Mercedes и Renault соответственно. Наибольшая путаница наблюдается между Ferrari и Mercedes (14% объектов Ferrari отнесены к Mercedes), что объясняется визуальным сходством некоторых моделей: жёлтые и красные купе обеих марок имеют схожую форму кузова, и модель может опираться на цвет как на ложный признак. Класс Renault распознаётся наиболее уверенно (recall 0.85, precision 0.83) – вероятно, благодаря характерным для марки кроссоверам и хетчбэкам, заметно отличающимся от спорткаров Ferrari.

Метрики classification_report: accuracy на тестовой выборке составила 0.83, средневзвешенный F1-score – 0.83, что согласуется с результатом на проверочной выборке (85.90%). Небольшой разрыв между val_accuracy и test_accuracy находится в пределах статистической погрешности при объёме тестовой выборки 343 изображения и подтверждает, что модель не переобучилась.

3.15 Визуальный просмотр предсказаний.

Для качественного анализа работы модели выводится сетка из 10 случайных изображений тестовой выборки с указанием истинного и предсказанного класса, а также уверенности модели. Заголовки правильных предсказаний окрашены в зелёный цвет, ошибочных – в красный, что наглядно демонстрирует характер допускаемых ошибок.

```
In [16]: # Просмотр предсказаний на случайных примерах
fig, axs = plt.subplots(2, 5, figsize=(20, 8))
axs = axs.flatten()

indices = random.sample(range(len(x_test)), 10)
for i, idx in enumerate(indices):
    img = x_test[idx].astype('uint8')
    true_label = CLASS_LIST[y_test[idx]]
    pred_label = CLASS_LIST[y_pred_classes[idx]]
    pred_proba = y_pred[idx][y_pred_classes[idx]]

    color = 'green' if true_label == pred_label else 'red'
    axs[i].imshow(img)
    axs[i].set_title(f'Истина: {true_label}\nПредсказано: {pred_label} ({pred_proba:.2f})',
                     color=color, fontsize=10)
    axs[i].axis('off')

plt.tight_layout()
plt.show()
```

Рисунок 3.15.1 – Код и примеры предсказаний модели на случайных изображениях

На приведённой выборке 9 из 10 изображений классифицированы правильно, что соответствует общей точности модели. Видно, что модель уверенно работает на типичных рекламных и студийных снимках (уверенность 0.97–1.00) и менее уверенно – на сложных кадрах с необычным ракурсом или при наличии других объектов в кадре (тёмный Renault сзади ошибочно отнесён к Mercedes с уверенностью 0.57; жёлтый Ferrari классифицирован правильно, но всего с уверенностью 0.63). Невысокая уверенность ошибочных предсказаний (около 0.55–0.65) – хороший признак: модель «честно» сигнализирует о неуверенности в сложных случаях, что в практическом применении позволяет настроить порог отсечения и отправлять такие случаи на ручную проверку.

Выводы: в рамках задания была построена и обучена модель классификации автомобилей по маркам (Ferrari, Mercedes, Renault) на основе технологии Transfer Learning с использованием предобученной MobileNetV2 в качестве базы. Обучение проводилось в два этапа: сначала обучалась только новая «голова» при замороженной базе (10 эпох, learning rate 0.001), затем выполнялся fine-tuning верхних 30% слоёв базы с малым learning rate (1e-5) и более терпеливыми колбэками.

Лучшая точность на проверочной выборке составила 85.90%, на тестовой – 83%. Целевой порог 93% не был достигнут, однако полученный результат демонстрирует ключевые принципы Transfer Learning: уже после первой эпохи val_accuracy составила 0.80, что значительно превышает результат при обучении с нуля, и подтверждает преимущество переноса знаний с большого датасета (ImageNet) на малую целевую задачу.

Анализ ошибок показал, что основная путаница происходит между визуально схожими моделями (например, между Ferrari и Mercedes у спорткаров с яркой окраской), и невысокая уверенность ошибочных предсказаний свидетельствует о корректной откалиброванности softmax-вероятностей. Для достижения целевой точности 93% потребовалось бы увеличить продолжительность fine-tuning, ослабить регуляризацию или использовать более мощную предобученную базу – эти направления могут стать предметом дальнейшего экспериментирования.

Общие выводы: в ходе лабораторной работы были изучены архитектура и принципы работы свёрточных нейронных сетей и приобретены практические навыки построения CNN на различных задачах компьютерного зрения с помощью двух популярных фреймворков – Keras и PyTorch.